

**ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH**



**BÁO CÁO
ĐỒ ÁN TỐT NGHIỆP**

**NGHIÊN CỨU VÀ PHÁT TRIỂN ROBOT TRỢ LÝ ẢO
ỨNG DỤNG TRÍ TUỆ NHÂN TẠO HỖ TRỢ CHĂM SÓC
VÀ GIÁM SÁT NGƯỜI CAO TUỔI TẠI NHÀ**

Ngành: Kỹ thuật Máy tính

HỘI ĐỒNG: 10L

GVHD: TS. Lê Trọng Nhân

GVPB: PGS. TS. Trần Văn Hoài

—o0o—

SVTH 1: Lê Đăng Khoa (2211601)

SVTH 2: Vũ Trọng Khánh (2211541)

SVTH 3: Lương Văn Khanh (2211486)

Tp. Hồ Chí Minh, tháng 12/2025

LỜI CAM ĐOAN

Chúng em xin cam đoan rằng đề án với đề tài “*Nghiên cứu và phát triển robot trợ lý ảo ứng dụng trí tuệ nhân tạo hỗ trợ chăm sóc và giám sát người cao tuổi tại nhà*” là công trình nghiên cứu do chính chúng em thực hiện dưới sự hướng dẫn khoa học của TS. Lê Trọng Nhân. Toàn bộ nội dung của đề án được xây dựng trên cơ sở quá trình học tập, nghiên cứu nghiêm túc, có sự tìm hiểu, phân tích và tổng hợp từ nhiều nguồn tài liệu khác nhau.

Chúng em xin cam đoan rằng các số liệu, kết quả thực nghiệm, mô hình đề xuất, hình ảnh minh họa và các nội dung trình bày trong đề án là trung thực, phản ánh đúng quá trình nghiên cứu và thực hiện đề tài.

Trong quá trình thực hiện đề án, chúng em có tham khảo và sử dụng các tài liệu, công trình nghiên cứu, sách, bài báo khoa học, cũng như các nguồn thông tin hợp pháp khác. Tất cả các tài liệu tham khảo đều đã được trích dẫn đầy đủ, chính xác và tuân thủ đúng các quy định về trích dẫn học thuật. Chúng em xin cam đoan không sao chép, không đạo văn, không sử dụng trái phép kết quả nghiên cứu của các tác giả khác dưới bất kỳ hình thức nào.

Chúng em xin hoàn toàn chịu trách nhiệm trước Khoa, Nhà trường và Hội đồng chấm đề án về tính trung thực, chính xác và tính pháp lý của toàn bộ nội dung được trình bày trong đề án này. Nếu có bất kỳ sai sót hoặc vi phạm nào liên quan đến bản quyền, đạo đức học thuật hoặc tính xác thực của dữ liệu, chúng em xin hoàn toàn chịu trách nhiệm theo các quy định hiện hành.

Nhóm tác giả

Lê Đăng Khoa

Vũ Trọng Khánh

Lương Văn Khanh

LỜI CẢM ƠN

Trước hết, chúng em xin bày tỏ lòng biết ơn sâu sắc đến TS. Lê Trọng Nhân, người đã trực tiếp hướng dẫn, định hướng và đồng hành cùng chúng em trong suốt quá trình thực hiện đề tài “*Nghiên cứu và phát triển robot trợ lý ảo ứng dụng trí tuệ nhân tạo hỗ trợ chăm sóc và giám sát người cao tuổi tại nhà*”. Trong quá trình thực hiện đồ án, thầy đã tận tình chỉ dẫn về phương pháp nghiên cứu, định hướng triển khai nội dung, cũng như thường xuyên góp ý, chỉnh sửa và giải đáp những khó khăn mà chúng em gặp phải. Sự hướng dẫn tận tâm và những ý kiến chuyên môn quý báu của thầy đã giúp chúng em từng bước hoàn thiện đề tài theo hướng khoa học, thực tiễn và có giá trị ứng dụng.

Chúng em xin chân thành cảm ơn quý thầy cô trong Khoa/Viện đã giảng dạy và truyền đạt cho chúng em những kiến thức nền tảng và chuyên môn trong suốt quá trình học tập. Những kiến thức và kinh nghiệm mà quý thầy cô đã chia sẻ không chỉ giúp chúng em hoàn thành đồ án này mà còn là hành trang quan trọng cho quá trình học tập và công tác sau này.

Bên cạnh đó, chúng em xin gửi lời cảm ơn đến Khoa và Nhà trường đã tạo điều kiện thuận lợi về cơ sở vật chất, môi trường học tập và nghiên cứu, giúp chúng em có thể triển khai và hoàn thành đề tài đúng tiến độ.

Chúng em cũng xin chân thành cảm ơn gia đình và bạn bè đã luôn quan tâm, động viên, chia sẻ và hỗ trợ chúng em cả về tinh thần lẫn thời gian trong suốt quá trình học tập và thực hiện đồ án. Sự động viên đó là nguồn khích lệ to lớn giúp chúng em vượt qua những khó khăn và áp lực trong quá trình nghiên cứu.

Mặc dù đã có nhiều cố gắng và nỗ lực trong quá trình thực hiện, song do hạn chế về thời gian, kiến thức và kinh nghiệm nghiên cứu, đồ án không tránh khỏi những thiếu sót. Chúng em rất mong nhận được sự góp ý và chỉ dẫn từ quý thầy cô để đề tài được hoàn thiện hơn.

Chúng em xin chân thành cảm ơn.

Nhóm tác giả

Lê Đăng Khoa

Vũ Trọng Khánh

Lương Văn Khanh

TÓM TẮT

Trong bối cảnh già hóa dân số và nhu cầu chăm sóc người cao tuổi tại nhà ngày càng tăng. Việc giám sát an toàn, duy trì liên lạc và hỗ trợ sinh hoạt hằng ngày cho người cao tuổi đặt ra nhiều khó khăn với các gia đình có người chăm sóc thường xuyên vắng mặt. Đồ án nghiên cứu và xây dựng hệ thống robot trợ lý ảo ứng dụng trí tuệ nhân tạo (Artificial Intelligence – AI), sử dụng thiết bị Android gắn trên robot làm bộ não xử lý. Hệ thống hướng đến việc tăng khả năng hỗ trợ kịp thời khi xảy ra sự cố, tạo phương thức tương tác thuận tiện bằng giọng nói cho người cao tuổi và giúp người chăm sóc theo dõi từ xa hiệu quả hơn.

Giải pháp gồm hai ứng dụng Android. Ứng dụng Robot khai thác camera để xử lý thị giác máy tính tại biên. Chuỗi xử lý kết hợp mô hình ước lượng tư thế You Only Look Once (YOLO), thuật toán Simple Online and Realtime Tracking (SORT) và đặc trưng tái định danh để theo dõi, điều khiển robot tự động di chuyển theo người cần chăm sóc; mô hình Long Short-Term Memory (LSTM) cùng bước hậu kiểm được sử dụng để nhận biết tình huống nghi té ngã. Khi phát hiện dấu hiệu bất thường, robot chủ động hỏi thăm bằng giọng nói để xác nhận tình trạng của người dùng; trong trường hợp người dùng cần trợ giúp hoặc không có phản hồi, hệ thống gửi cảnh báo và thiết lập cuộc gọi đến người chăm sóc. Bên cạnh đó, robot có khả năng trò chuyện, giúp nhắc nhở và điều khiển qua giọng nói, qua đó hỗ trợ một số nhu cầu giao tiếp và sinh hoạt thường ngày. Ứng dụng Caregiver cho phép người chăm sóc quản lý hồ sơ, thiết lập các nhắc nhở, xem nhật ký các nhắc nhở, cảnh báo té ngã và cuộc gọi; đồng thời có thể bật hoặc tắt các chức năng giám sát, gọi video hai chiều và điều khiển robot bằng cần điều khiển ảo.

Kết quả của đồ án là một nguyên mẫu tích hợp hai ứng dụng di động, khối xử lý trí tuệ nhân tạo, hạ tầng kết nối và phần cứng robot. Là sự tận dụng năng lực tính toán và cảm biến của thiết bị Android để giảm yêu cầu về phần cứng chuyên dụng. Là tiền đề cho việc tiếp tục nghiên cứu và phát triển các ứng dụng, sản phẩm công nghệ hỗ trợ chăm sóc người cao tuổi tại nhà. Hệ thống có thể được hoàn thiện thêm về độ tin cậy, bảo mật, khả năng ghép cặp nhiều thiết bị và các chức năng theo dõi sức khỏe, qua đó góp phần mở rộng khả năng ứng dụng trí tuệ nhân tạo và robot trong lĩnh vực chăm sóc người cao tuổi.

Từ khóa: robot trợ lý, chăm sóc người cao tuổi, phát hiện té ngã, thị giác máy tính, xử lý tiếng nói, Android, Internet vạn vật.

MỤC LỤC

Lời cam đoan	i
Lời cảm ơn	ii
Tóm tắt	iii
Mục lục	iv
Danh mục hình ảnh	vii
Danh mục bảng biểu	viii
Danh mục từ viết tắt	ix
1 Giới thiệu	1
1.1 Tính cấp thiết của đề tài	1
1.2 Mục tiêu và phạm vi đề tài	1
1.2.1 Mục tiêu tổng quát	1
1.2.2 Mục tiêu cụ thể	2
1.2.3 Phạm vi đề tài	2
1.3 Đối tượng nghiên cứu	2
1.4 Phương pháp nghiên cứu	3
1.4.1 Phương pháp nghiên cứu lý thuyết	3
1.4.2 Phương pháp nghiên cứu thực nghiệm	3
1.5 Các công trình nghiên cứu liên quan	3
2 Cơ sở lý thuyết	5
2.1 Nền tảng Intel OpenBot	5
2.1.1 Giới thiệu Intel OpenBot	5
2.1.2 Kiến trúc hệ thống OpenBot	5
2.1.3 Giao tiếp giữa Smartphone và Robot	6
2.2 Tổng quan về Mạng Nơ-ron Nhân tạo (Neural Network)	7
2.2.1 Khái niệm và nguồn gốc sinh học	7
2.2.2 Cấu trúc neuron nhân tạo	7
2.2.3 Lan truyền ngược (Backpropagation)	9

2.2.4	Vai trò trong Thị giác Máy tính	13
2.3	Mạng Perceptron Đa tầng (MLP)	16
2.3.1	Định nghĩa MLP	16
2.3.2	Cấu trúc nhiều lớp	16
2.3.3	Ưu điểm, hạn chế của MLP và giới hạn trong xử lý ảnh	16
2.4	Mạng Nơ-ron Tích chập (CNN)	18
2.4.1	Động cơ ra đời của CNN	18
2.4.2	Các thành phần chính của CNN	20
2.4.3	Ưu điểm của CNN trong xử lý ảnh và video	21
2.4.4	Các kiến trúc CNN tiêu biểu	21
2.5	Giới thiệu YOLO (You Only Look Once)	23
2.5.1	Bài toán Object Detection	23
2.5.2	Lịch sử phát triển Object Detection	23
2.5.3	Khái niệm Bounding Box	24
2.5.4	Intersection over Union (IoU)	26
2.5.5	Các khái niệm True Positive, False Positive, False Negative	26
2.5.6	Đánh giá hiệu suất: Mean Average Precision (mAP)	27
2.5.7	Ý tưởng YOLO: Phát hiện trong một lần suy luận	28
2.5.8	So sánh YOLO với phương pháp truyền thống	29
2.6	Lý do lựa chọn YOLOv11 cho bài toán Fall Detection	30
2.6.1	Giới thiệu YOLOv11	30
2.6.2	Các phiên bản mô hình YOLOv11	31
2.7	Theo dõi và Nhận diện Người (Person Tracking)	31
2.7.1	Giới thiệu Person Tracking	31
2.7.2	Tracking-by-Detection Paradigm	32
2.8	Giới thiệu Dataset Fall Detection – CauCaFall	33
2.8.1	Mục tiêu và đặc điểm của dataset CauCaFall	33
2.8.2	Dữ liệu video trong nhà	33
2.8.3	Annotation ban đầu	34
2.8.4	Ưu điểm và hạn chế của dataset CauCaFall	34
2.8.5	Nghiên cứu liên quan: Dataset DiverseFALL10500	35
2.9	Hệ thống AI Agent và Mô hình Ngôn ngữ Lớn (LLM)	36
2.9.1	Khái niệm AI Agent	36
2.9.2	Mô hình Ngôn ngữ Lớn (Large Language Models)	36

2.9.3	Retrieval-Augmented Generation (RAG)	44
2.10	Hệ thống Nhận dạng Giọng nói và Xử lý Ngôn ngữ Tự nhiên	45
2.10.1	Một số giải pháp Nhận dạng Giọng nói:	45
2.10.2	So sánh các phương pháp Xử lý Ngôn ngữ Tự nhiên (NLP)	45
2.10.3	Phương thức triển khai: Cloud Computing và Edge Computing	46
2.11	Kết luận chương	46
3	Phân tích và thiết kế hệ thống	48
3.1	Đề xuất thiết kế hệ thống	48
3.2	Đề xuất giải pháp cho chức năng phát hiện té ngã	51
3.2.1	Lựa chọn YOLOv11 cho chức năng phát hiện té ngã	51
3.2.2	Lựa chọn YOLOv11n cho Robot	52
3.3	Đề xuất giải pháp cho chức năng theo dõi(tracking)	53
3.3.1	Các thuật toán Tracking phổ biến	53
3.3.2	Chi tiết thuật toán ByteTrack	54
3.3.3	Person Re-identification (ReID)	56
3.4	Đề xuất giải pháp cho chức năng xử lý ngôn ngữ, giao tiếp	57
3.5	Đề xuất các giao thức giao tiếp sử dụng	57
3.5.1	Giao thức MQTT (Message Queuing Telemetry Transport)	57
3.5.2	Giao thức WebRTC (Real-Time Communication)	58
3.5.3	Giao thức HTTP/HTTPS	58
3.5.4	Giao tiếp nội bộ (Serial/Bluetooth)	58
3.5.5	Kết nối Internet (TCP/IP)	59
3.6	Vấn đề liên quan mất đồng bộ giữa tín hiệu điều khiển Robot và Video call	59
4	Hiện thực hệ thống	63
5	Kết quả và thảo luận	64
6	Kết luận	65
6.1	Nhận xét	65
6.2	Hướng phát triển	65
	Tài liệu tham khảo	66

DANH SÁCH HÌNH VẼ

2.1	So sánh giữa neuron sinh học (trái) và neuron nhân tạo (phải). Các thành phần tương ứng: dendrite $\leftrightarrow$ input, synapse $\leftrightarrow$ weight, soma $\leftrightarrow$ summation, axon $\leftrightarrow$ activation function và output.	8
2.2	Computational Graph minh họa quá trình Forward Pass (mũi tên xanh) và Backward Pass (mũi tên đỏ) với các gradient được tính ngược từ Loss về các tham số.	10
2.3	Bề mặt hàm mất mát $J(\theta_0, \theta_1)$ và quỹ đạo của Gradient Descent (đường đen) tìm điểm cực tiểu.	12
2.4	Bốn bài toán chính trong Thị giác Máy tính: Phân loại ảnh, Phát hiện đối tượng, Phân vùng ngữ nghĩa, và Nhận dạng hành động.	13
2.5	Kiến trúc mạng Perceptron Đa tầng (MLP) với lớp đầu vào, hai lớp ẩn và lớp đầu ra. Mỗi neuron trong một lớp kết nối đầy đủ với tất cả neuron của lớp liền kề.	17
2.6	Minh họa phép tích chập (Convolution) và Max Pooling trong CNN: bộ lọc 3×3 trượt qua ảnh đầu vào tạo ra feature map, sau đó Max Pooling 2×2 giảm kích thước không gian.	20
2.7	AlexNet	22
2.8	VGG	22
2.9	Ví dụ minh họa Bounding Box trong Object Detection.	25
2.10	Minh họa chia lưới 7×7 trong YOLO: mỗi ô chịu trách nhiệm phát hiện đối tượng có tâm nằm trong ô đó.	29
2.11	Flowchart pipeline Tracking-by-Detection: Detection $\rightarrow$ Feature Extraction $\rightarrow$ Association $\rightarrow$ State Update, lặp lại cho mỗi frame.	32
2.12	Minh họa không gian vector embedding và độ tương đồng cosine giữa các từ	37
2.13	Kiến trúc Transformer với các khối Encoder và Decoder	38
2.14	Cơ chế Self-Attention: Query, Key, Value và Scaled Dot-Product Attention	39
2.15	Ví dụ Chain-of-Thought: thay vì trả lời trực tiếp, mô hình giải thích từng bước suy luận để đi đến kết quả cuối cùng.	44
3.1	Thiết kế hệ thống đề xuất	50

DANH SÁCH BẢNG

2.1	Ưu điểm và hạn chế của mạng MLP	17
2.2	So sánh YOLO với các phương pháp Object Detection khác	30
2.3	So sánh Transformer và CNN	40
2.4	So sánh các công nghệ Speech-to-Text.	45
2.5	So sánh các phương pháp NLP.	45
3.1	So sánh các thuật toán Multi-Object Tracking	53
3.2	So sánh các mô hình Person Re-identification	56

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Thuật ngữ tiếng Anh	Ý nghĩa tiếng Việt
AI	Artificial Intelligence	Trí tuệ nhân tạo
AP	Average Precision	Độ chính xác trung bình
API	Application Programming Interface	Giao diện lập trình ứng dụng
BERT	Bidirectional Encoder Representations from Transformers	Biểu diễn mã hóa hai chiều từ Transformer
BLE	Bluetooth Low Energy	Bluetooth năng lượng thấp
CNN	Convolutional Neural Network	Mạng nơ-ron tích chập
COCO	Common Objects in Context	Bộ dữ liệu đối tượng phổ biến trong ngữ cảnh
CPU	Central Processing Unit	Bộ xử lý trung tâm
FFN	Feed-Forward Network	Mạng nơ-ron truyền thẳng
FPS	Frames Per Second	Số khung hình trên giây
GPU	Graphics Processing Unit	Bộ xử lý đồ họa
GPT	Generative Pre-trained Transformer	Mô hình Transformer sinh được tiền huấn luyện
IoT	Internet of Things	Internet vạn vật
IoU	Intersection over Union	Tỷ lệ phần giao trên phần hợp
LLM	Large Language Model	Mô hình ngôn ngữ lớn
LSTM	Long Short-Term Memory	Mạng bộ nhớ dài-ngắn hạn
MLP	Multilayer Perceptron	Mạng perceptron đa lớp
MOTA	Multiple Object Tracking Accuracy	Độ chính xác theo dõi đa đối tượng
MQTT	Message Queuing Telemetry Transport	Giao thức truyền thông điệp nhẹ
NAT	Network Address Translation	Biên dịch địa chỉ mạng
NLP	Natural Language Processing	Xử lý ngôn ngữ tự nhiên
NLU	Natural Language Understanding	Hiểu ngôn ngữ tự nhiên

Từ viết tắt	Thuật ngữ tiếng Anh	Ý nghĩa tiếng Việt
NMS	Non-Maximum Suppression	Úc chế không cực đại
P2P	Peer-to-Peer	Kết nối ngang hàng
R-CNN	Region-based Convolutional Neural Network	Mạng nơ-ron tích chập dựa trên vùng
RAG	Retrieval-Augmented Generation	Sinh nội dung có tăng cường truy xuất
Re-ID	Person Re-identification	Tái định danh người
RGB	Red, Green, Blue	Hệ màu đỏ, xanh lá và xanh dương
SORT	Simple Online and Realtime Tracking	Thuật toán theo dõi trực tuyến thời gian thực
SSD	Single Shot MultiBox Detector	Mô hình phát hiện đối tượng một giai đoạn
SOTA	State of the Art	Trình độ tiên tiến nhất tại thời điểm xem xét
STT	Speech-to-Text	Chuyển giọng nói thành văn bản
TTS	Text-to-Speech	Chuyển văn bản thành giọng nói
WebRTC	Web Real-Time Communication	Giao tiếp thời gian thực trên nền web
WHO	World Health Organization	Tổ chức Y tế Thế giới
YOLO	You Only Look Once	Họ mô hình thị giác máy tính thời gian thực

Chương 1 – Giới thiệu

1.1 Tính cấp thiết của đề tài

Trong bối cảnh xã hội hiện đại, vấn đề già hóa dân số đang trở thành một thách thức lớn đối với nhiều quốc gia trên thế giới, trong đó có Việt Nam. Theo báo cáo của Tổng cục Thống kê, Việt Nam đã chính thức bước vào giai đoạn già hóa dân số từ năm 2011, với tỷ lệ người từ 60 tuổi trở lên ngày càng tăng nhanh. Dự báo đến năm 2050, tỷ lệ người cao tuổi sẽ chiếm khoảng 25% dân số cả nước. Sự gia tăng này đặt ra nhiều vấn đề về chăm sóc sức khỏe, an sinh xã hội và đặc biệt là nhu cầu hỗ trợ người cao tuổi trong sinh hoạt hàng ngày.

Người cao tuổi thường phải đối mặt với nhiều thách thức trong cuộc sống như: suy giảm trí nhớ dẫn đến quên uống thuốc đúng giờ, khả năng vận động hạn chế làm tăng nguy cơ té ngã, cảm giác cô đơn khi sống một mình hoặc con cháu bận rộn công việc. Theo thống kê của Tổ chức Y tế Thế giới (WHO), té ngã là nguyên nhân gây tử vong đứng thứ hai ở người cao tuổi, với khoảng 37.3 triệu ca té ngã nghiêm trọng mỗi năm trên toàn cầu. Bên cạnh đó, việc không tuân thủ lịch uống thuốc có thể dẫn đến các biến chứng nghiêm trọng về sức khỏe.

Trong khi đó, sự phát triển mạnh mẽ của công nghệ trí tuệ nhân tạo (AI), Internet vạn vật (IoT), và thị giác máy tính đã mở ra nhiều cơ hội ứng dụng trong lĩnh vực chăm sóc sức khỏe. Các hệ thống robot thông minh với khả năng giao tiếp bằng giọng nói, nhận dạng hình ảnh, và kết nối với các thiết bị IoT có thể đóng vai trò như một trợ lý ảo, hỗ trợ người cao tuổi trong các hoạt động thường ngày một cách hiệu quả và an toàn.

Từ những thực tế trên, đề tài **”Nghiên cứu và phát triển robot trợ lý ảo ứng dụng trí tuệ nhân tạo hỗ trợ chăm sóc và giám sát người cao tuổi tại nhà”** được thực hiện nhằm xây dựng một hệ thống robot thông minh, tích hợp các công nghệ AI tiên tiến để hỗ trợ toàn diện cho người cao tuổi, góp phần nâng cao chất lượng cuộc sống và giảm bớt gánh nặng cho người thân cũng như hệ thống y tế.

1.2 Mục tiêu và phạm vi đề tài

1.2.1 Mục tiêu tổng quát

Xây dựng một hệ thống robot trợ lý ảo có khả năng giao tiếp bằng giọng nói dựa trên công nghệ trí tuệ nhân tạo (AI), hỗ trợ nhắc nhở uống thuốc, giám sát hành vi, cảnh

bảo tể ngã và hỗ trợ các hoạt động thường ngày, nhằm nâng cao chất lượng cuộc sống cho người cao tuổi tại nhà.

1.2.2 Mục tiêu cụ thể

- Tìm hiểu nền tảng Intel OpenBot cùng hệ thống giao tiếp giữa điện thoại thông minh và phần cứng robot.
- Phát triển hệ thống nhận dạng giọng nói tích hợp với khả năng hiểu ngôn ngữ tự nhiên (NLU - Natural Language Understanding) để diễn giải các lệnh từ người dùng.
- Triển khai hệ thống trí tuệ nhân tạo cho giao tiếp thông minh và hệ thống kết nối vạn vật (IoT) cho xử lý, điều khiển, tích hợp.
- Nghiên cứu và ứng dụng thị giác máy tính (Computer Vision) và trí tuệ nhân tạo cho hệ thống giám sát, theo dõi, phân tích và xử lý.
- Đánh giá về khả năng và tốc độ xử lý của các thuật toán trí tuệ nhân tạo và hệ thống xử lý, kết nối.

1.2.3 Phạm vi đề tài

- **Phạm vi không gian:** Hệ thống được thiết kế và triển khai trong môi trường nhà ở, tập trung vào các khu vực sinh hoạt chính của người cao tuổi.
- **Phạm vi đối tượng:** Người cao tuổi (từ 60 tuổi trở lên) có sức khỏe yếu cần được giám sát, đặc biệt là những người sống một mình hoặc có người thân bận rộn.
- **Phạm vi chức năng:** Tập trung vào các chức năng cốt lõi bao gồm: giao tiếp bằng giọng nói, nhắc nhở uống thuốc, phát hiện và cảnh báo té ngã, giám sát hoạt động.

1.3 Đối tượng nghiên cứu

- **Nền tảng Intel OpenBot:** Tìm hiểu kiến trúc phần cứng và phần mềm của nền tảng robot mã nguồn mở Intel OpenBot, cơ chế giao tiếp giữa điện thoại thông minh và phần cứng robot.
- **Công nghệ nhận dạng và xử lý giọng nói:** Các kỹ thuật Speech-to-Text (STT), Text-to-Speech (TTS), và xử lý ngôn ngữ tự nhiên (NLP).

- **Mô hình ngôn ngữ lớn (LLM):** Nghiên cứu và tích hợp các mô hình ngôn ngữ lớn để tạo khả năng giao tiếp thông minh, hiểu ngữ cảnh và đưa ra phản hồi phù hợp.
- **Thị giác máy tính:** Các thuật toán phát hiện đối tượng (Object Detection), theo dõi người (Person Tracking), và nhận dạng hành động (Action Recognition) để phát hiện té ngã.
- **Internet vạn vật (IoT):** Các giao thức và nền tảng IoT để kết nối, điều khiển Robot, nhận và gửi dữ liệu từ xa thông qua ứng dụng di động.

1.4 Phương pháp nghiên cứu

1.4.1 Phương pháp nghiên cứu lý thuyết

- Thu thập, tổng hợp và phân tích các tài liệu, công trình nghiên cứu liên quan đến robot hỗ trợ người cao tuổi, công nghệ AI, thị giác máy tính và IoT.
- Nghiên cứu các mô hình học sâu (Deep Learning) tiên tiến trong lĩnh vực xử lý ngôn ngữ tự nhiên và thị giác máy tính.
- Tìm hiểu các tiêu chuẩn và yêu cầu về an toàn, bảo mật trong các hệ thống chăm sóc sức khỏe.

1.4.2 Phương pháp nghiên cứu thực nghiệm

- Có thể tiến hành mô phỏng, hiện thực hệ thống từ đó thu thập thông tin để đánh giá, so sánh để đi đến lựa chọn phương án hiện thực triển khai hệ thống.
- Kiểm thử và đánh giá hiệu năng hệ thống trong môi trường thực tế.

1.5 Các công trình nghiên cứu liên quan

Trong những năm gần đây, đã có nhiều công trình nghiên cứu và sản phẩm thương mại hướng đến việc hỗ trợ người cao tuổi sử dụng công nghệ robot và AI:

- **Robot Pepper (SoftBank Robotics):** Pepper là robot hình người được phát triển bởi SoftBank Robotics, có khả năng nhận dạng cảm xúc thông qua giọng nói và biểu cảm khuôn mặt, đồng thời hỗ trợ giao tiếp tự nhiên với con người. Robot này đã và đang được triển khai trong lĩnh vực chăm sóc người cao tuổi tại Nhật Bản nhằm hỗ trợ tương tác xã hội và cải thiện chất lượng cuộc sống.



- **Robot ElliQ:** Một robot bàn nhỏ gọn được thiết kế đặc biệt cho người cao tuổi giúp họ chống lại cảm giác cô đơn và cô lập. Nó có khả năng nhắc nhở uống thuốc, gợi ý hoạt động và kết nối với người thân.



- **Các nghiên cứu về Fall Detection:** Nhiều công trình sử dụng mạng nơ-ron tích chập (CNN), đặc biệt là các mô hình YOLO, để phát hiện té ngã thời gian thực với độ chính xác cao.

Tuy nhiên, các giải pháp hiện có thường có chi phí cao, khó tiếp cận với đại đa số người dùng Việt Nam, hoặc chưa được tối ưu hóa cho ngữ cảnh và ngôn ngữ tiếng Việt. Đề tài này hướng đến việc xây dựng một giải pháp toàn diện, tích hợp nhiều công nghệ AI tiên tiến, với chi phí hợp lý và khả năng tùy biến cao, phù hợp với nhu cầu thực tế của người cao tuổi Việt Nam.

Chương 2 – Cơ sở lý thuyết

Phần này trình bày nền tảng lý thuyết cần thiết cho hệ thống robot thông minh tích hợp trí tuệ nhân tạo phục vụ giám sát và phát hiện té ngã. Nội dung được tổ chức theo thứ tự: tìm hiểu nền tảng Intel OpenBot; hệ thống nhận dạng giọng nói và xử lý ngôn ngữ tự nhiên; công nghệ Internet vạn vật (IoT) cho kết nối và điều khiển; các kiến trúc mạng nơ-ron nhân tạo (MLP, CNN); mô hình YOLO và lý do lựa chọn YOLOv11; cùng phương pháp xây dựng tập dữ liệu đa nhiệm phục vụ huấn luyện mô hình phát hiện ngã.

2.1 Nền tảng Intel OpenBot

2.1.1 Giới thiệu Intel OpenBot

Intel OpenBot là một nền tảng robot mã nguồn mở được Intel phát triển nhằm mục đích nghiên cứu và giáo dục trong lĩnh vực robotics và trí tuệ nhân tạo. Điểm đặc biệt của OpenBot là sử dụng điện thoại thông minh (smartphone) làm “bộ não” chính của robot, tận dụng sức mạnh tính toán, camera, và các cảm biến có sẵn trên điện thoại.

Các đặc điểm nổi bật của Intel OpenBot:

- **Chi phí thấp:** Phần cứng cơ bản chỉ khoảng 50-100 USD, bao gồm khung xe, động cơ, và mạch điều khiển đơn giản
- **Mã nguồn mở:** Toàn bộ thiết kế phần cứng (3D printable) và phần mềm (Android app, firmware) đều công khai
- **Tận dụng smartphone:** Sử dụng camera và khả năng kết nối của điện thoại để chạy các mô hình AI
- **Hỗ trợ Machine Learning:** Tích hợp TensorFlow Lite cho việc triển khai mô hình học sâu trực tiếp trên thiết bị

2.1.2 Kiến trúc hệ thống OpenBot

Hệ thống OpenBot bao gồm ba thành phần chính:

a) Phần cứng Robot (Hardware)

- **Khung xe:** Có thể in 3D hoặc sử dụng các kit robot có sẵn
- **Động cơ DC:** Thường sử dụng động cơ DC với encoder để điều khiển chính xác

- **Board điều khiển:** Arduino Nano hoặc ESP32 làm vi điều khiển
- **Cảm biến phụ trợ:** Có thể tích hợp thêm cảm biến siêu âm, IR, hoặc IMU
- **Pin:** Pin lithium để cấp nguồn cho robot và sạc điện thoại

b) Ứng dụng Android (Mobile App)

Ứng dụng Android là “bộ não” của robot, chịu trách nhiệm:

- Thu thập dữ liệu từ camera và các cảm biến của điện thoại
- Chạy các mô hình AI (object detection, navigation, policy learning)
- Gửi lệnh điều khiển đến phần cứng robot qua USB hoặc Bluetooth
- Giao diện người dùng cho việc điều khiển và giám sát

c) Firmware vi điều khiển

Firmware trên Arduino/ESP32 thực hiện các tác vụ cấp thấp:

- Nhận lệnh từ điện thoại (tốc độ, hướng đi)
- Điều khiển PWM cho động cơ
- Đọc encoder và cảm biến
- Gửi phản hồi trạng thái về điện thoại

2.1.3 Giao tiếp giữa Smartphone và Robot

Giao tiếp giữa điện thoại và phần cứng robot được thực hiện qua hai phương thức chính:

a) Kết nối USB Serial

- Sử dụng cáp USB OTG kết nối điện thoại với Arduino/ESP32
- Giao thức Serial với baudrate thường là 115200
- Ưu điểm: Độ trễ thấp, ổn định, có thể sạc pin cho điện thoại
- Nhược điểm: Dây cáp hạn chế tính linh hoạt

b) Kết nối Bluetooth

- Sử dụng module Bluetooth (HC-05/HC-06) hoặc Bluetooth tích hợp trên ESP32
- Giao thức Bluetooth Serial Port Profile (SPP)
- Ưu điểm: Không dây, linh hoạt
- Nhược điểm: Độ trễ cao hơn, cần quản lý pin riêng

2.2 Tổng quan về Mạng Nơ-ron Nhân tạo (Neural Network)

2.2.1 Khái niệm và nguồn gốc sinh học

Mạng nơ-ron nhân tạo (Artificial Neural Network - ANN) là một mô hình tính toán lấy cảm hứng từ cấu trúc và cách thức hoạt động của hệ thần kinh sinh học. Trong não bộ con người, hàng tỷ tế bào thần kinh (neuron) liên kết với nhau thông qua các khớp thần kinh (synapse) để xử lý và truyền tải thông tin. Mỗi neuron sinh học nhận tín hiệu đầu vào từ các neuron khác thông qua các sợi nhánh (dendrite), xử lý tín hiệu tại thân tế bào (soma), và truyền tín hiệu đầu ra qua sợi trục (axon) đến các neuron tiếp theo.

Dựa trên nguyên lý này, các nhà khoa học đã phát triển neuron nhân tạo như một đơn vị tính toán cơ bản. Ý tưởng đầu tiên về neuron nhân tạo được Warren McCulloch và Walter Pitts đề xuất vào năm 1943, mở đường cho sự phát triển của lĩnh vực học máy và trí tuệ nhân tạo hiện đại.

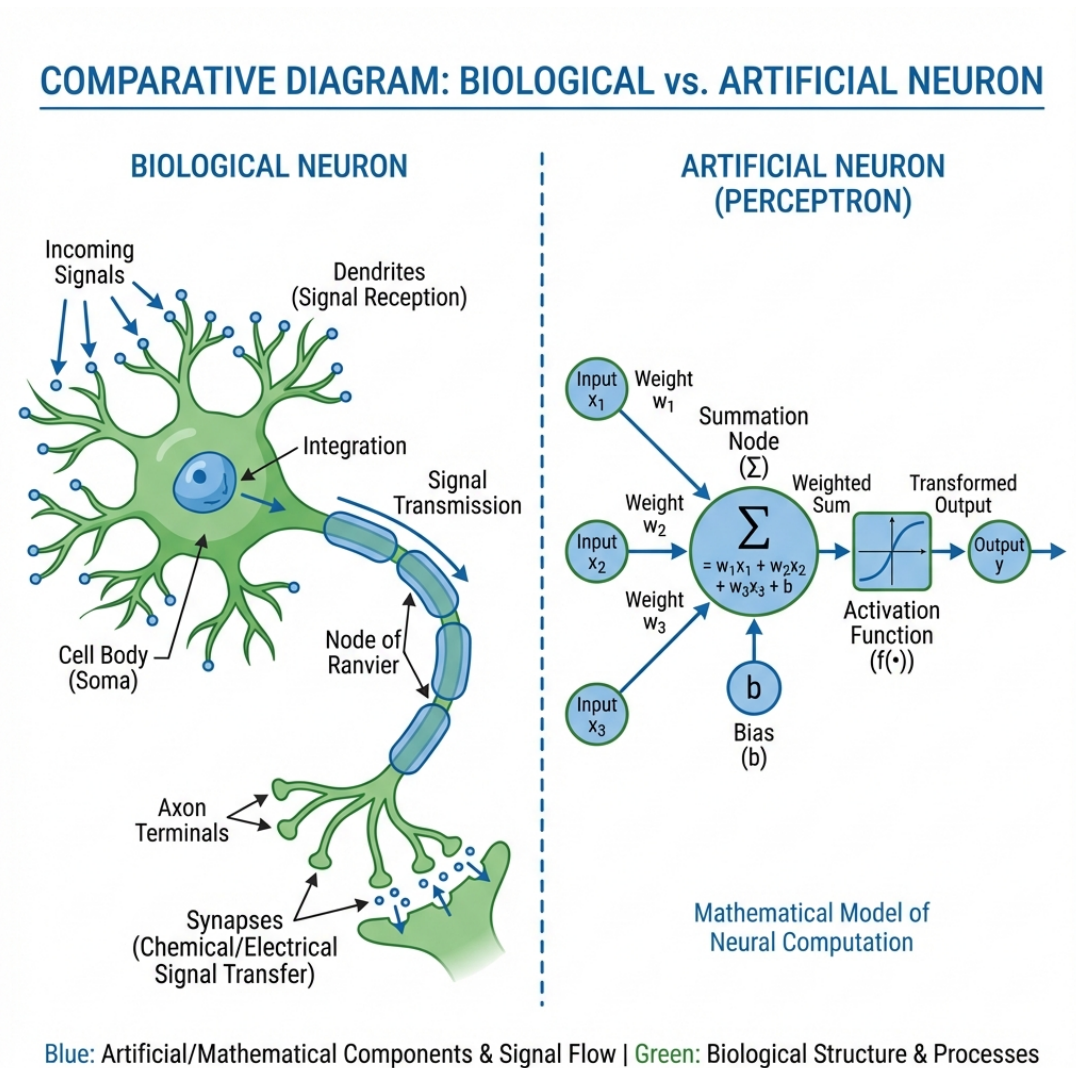
2.2.2 Cấu trúc neuron nhân tạo

Nguồn gốc sinh học và cảm hứng từ não bộ

Neuron nhân tạo được thiết kế dựa trên việc mô phỏng cấu trúc và chức năng của neuron sinh học trong hệ thần kinh con người. Một neuron sinh học hoạt động theo cơ chế:

1. **Tiếp nhận tín hiệu:** Các sợi nhánh (dendrite) tiếp nhận tín hiệu điện-hóa học từ hàng nghìn neuron khác thông qua các khớp thần kinh (synapse).
2. **Tích hợp thông tin:** Thân tế bào (soma) tổng hợp tất cả tín hiệu đầu vào. Mỗi synapse có “sức mạnh” khác nhau - một số kết nối có ảnh hưởng mạnh hơn (tương tự như trọng số trong mạng nhân tạo).

3. **Quyết định kích hoạt:** Nếu tổng tín hiệu vượt quá ngưỡng kích hoạt (threshold), neuron sẽ “bắn” (fire) một xung điện.
4. **Truyền tín hiệu:** Xung điện truyền dọc theo sợi trục (axon) đến các neuron tiếp theo.



Hình 2.1: So sánh giữa neuron sinh học (trái) và neuron nhân tạo (phải). Các thành phần tương ứng: dendrite ↔ input, synapse ↔ weight, soma ↔ summation, axon ↔ activation function và output.

Năm 1943, Warren McCulloch (nhà thần kinh học) và Walter Pitts (nhà logic học) đã công bố bài báo “A Logical Calculus of Ideas Immanent in Nervous Activity”, đề xuất mô hình toán học đầu tiên của neuron - đặt nền móng cho lĩnh vực mạng nơ-ron nhân tạo. Họ chứng minh rằng một mạng các neuron đơn giản có thể thực hiện bất kỳ phép tính logic nào.

Các thành phần của neuron nhân tạo

Ảnh xạ từ neuron sinh học sang nhân tạo, một neuron nhân tạo bao gồm các thành phần chính:

- **Đầu vào (Input):** Tập hợp các giá trị $x_1, x_2, \dots, x_n$ đại diện cho dữ liệu hoặc tín hiệu từ các neuron trước đó. (Tương ứng với *dendrite* trong sinh học)
- **Trọng số (Weight):** Mỗi đầu vào x_i được gắn với một trọng số w_i thể hiện mức độ quan trọng của đầu vào đó. Trọng số là tham số có thể học được trong quá trình huấn luyện. (Tương ứng với *sức mạnh synapse* - *sức mạnh kết nối giữa các neuron*)
- **Độ lệch (Bias):** Giá trị b được cộng thêm vào tổng có trọng số, cho phép neuron điều chỉnh ngưỡng kích hoạt một cách linh hoạt. (Tương ứng với *ngưỡng kích hoạt trong neuron sinh học*)
- **Hàm kích hoạt (Activation Function):** Hàm phi tuyến σ áp dụng lên tổng có trọng số để tạo ra đầu ra của neuron. (Mô phỏng cơ chế “bắn” hay không “bắn” của neuron sinh học)

Đầu ra của một neuron được tính theo công thức:

$$y = \sigma \left(\sum_{i=1}^n w_i x_i + b \right) = \sigma(W^T X + b) \quad (2.1)$$

Trong đó $W = [w_1, w_2, \dots, w_n]^T$ là vector trọng số, $X = [x_1, x_2, \dots, x_n]^T$ là vector đầu vào, và σ là hàm kích hoạt.

Các hàm kích hoạt phổ biến bao gồm:

- **Hàm Sigmoid:** $\sigma(z) = \frac{1}{1+e^{-z}}$, cho đầu ra trong khoảng $(0, 1)$
- **Hàm Tanh:** $\sigma(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$, cho đầu ra trong khoảng $(-1, 1)$
- **Hàm ReLU:** $\sigma(z) = \max(0, z)$, được sử dụng rộng rãi trong học sâu nhờ tính đơn giản và hiệu quả

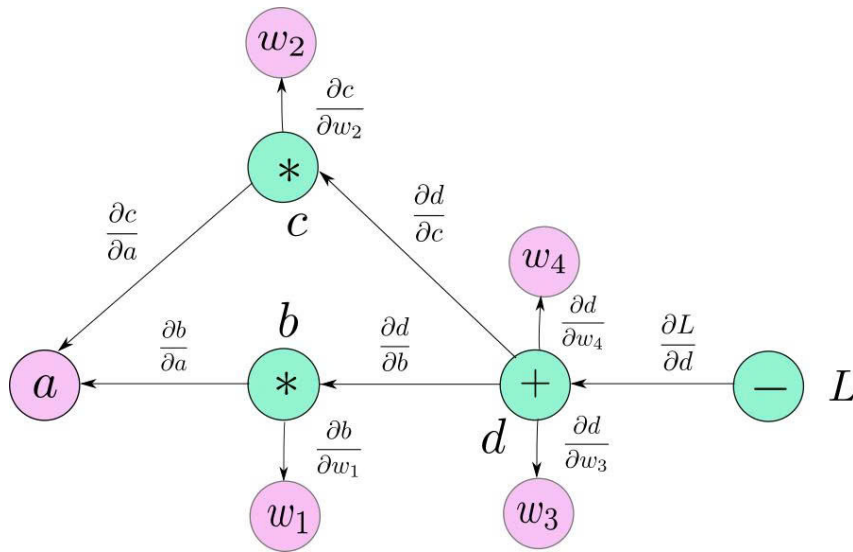
2.2.3 Lan truyền ngược (Backpropagation)

Lan truyền ngược là thuật toán cốt lõi để huấn luyện mạng nơ-ron, cho phép tính gradient của hàm mất mát theo từng tham số trong mạng. Thuật toán dựa trên quy tắc chuỗi (chain rule) trong giải tích.

Đồ thị Tính toán (Computational Graph)

Computational Graph là cách biểu diễn các phép tính toán học dưới dạng đồ thị có hướng, trong đó:

- **Các nút (nodes):** Đại diện cho các biến (inputs, outputs, intermediate values) hoặc các phép toán (cộng, nhân, activation function)
- **Các cạnh (edges):** Thể hiện sự phụ thuộc dữ liệu giữa các nút



Hình 2.2: Computational Graph minh họa quá trình Forward Pass (mũi tên xanh) và Backward Pass (mũi tên đỏ) với các gradient được tính ngược từ Loss về các tham số.

Computational Graph là nền tảng cho cơ chế **Automatic Differentiation** (vi phân tự động) trong các framework deep learning hiện đại như PyTorch, TensorFlow:

1. Forward Pass: Tính toán giá trị đầu ra theo chiều xuôi từ input đến loss

- Mỗi node lưu lại giá trị tính được (activation cache)
- Đồng thời xây dựng đồ thị các phép tính đã thực hiện

2. Backward Pass: Tính gradient theo chiều ngược từ loss về các tham số

- Bắt đầu từ $\frac{\partial \mathcal{L}}{\partial \mathcal{L}} = 1$
- Áp dụng chain rule tại mỗi node để lan truyền gradient ngược
- Mỗi node nhận gradient từ phía sau và tính gradient cho các input của nó

Ví dụ minh họa Chain Rule trên Computational Graph

Xét phép tính đơn giản: $\mathcal{L} = (w \cdot x + b)^2$ với $w = 2, x = 3, b = 1$

Forward Pass:

$$\begin{aligned} z1 &= w * x = 2 * 3 = 6 \\ z2 &= z1 + b = 6 + 1 = 7 \\ L &= z2^2 = 49 \end{aligned}$$

Backward Pass:

$$\begin{aligned} dL/dL &= 1 \\ dL/dz2 &= 2 * z2 = 14 \\ dL/dz1 &= dL/dz2 * dz2/dz1 = 14 * 1 = 14 \\ dL/db &= dL/dz2 * dz2/db = 14 * 1 = 14 \\ dL/dw &= dL/dz1 * dz1/dw = 14 * x = 14 * 3 = 42 \\ dL/dx &= dL/dz1 * dz1/dx = 14 * w = 14 * 2 = 28 \end{aligned}$$

Ưu điểm của Computational Graph:

- **Tự động hóa:** Không cần đạo hàm thủ công, framework tự động tính gradient
- **Hiệu quả:** Gradient được tính một lần backward pass, không cần tính lại nhiều lần
- **Linh hoạt:** Hỗ trợ dynamic computational graph (PyTorch) cho kiến trúc thay đổi
- **Tối ưu bộ nhớ:** Các framework áp dụng gradient checkpointing để tiết kiệm memory

Công thức Backpropagation

Cho hàm mất mát $\mathcal{L}$, gradient theo trọng số của lớp l được tính:

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \frac{\partial \mathcal{L}}{\partial a^{(l)}} \cdot \frac{\partial a^{(l)}}{\partial z^{(l)}} \cdot \frac{\partial z^{(l)}}{\partial W^{(l)}} \quad (2.2)$$

Đặt $\delta^{(l)} = \frac{\partial \mathcal{L}}{\partial z^{(l)}}$ là “error signal” của lớp l :

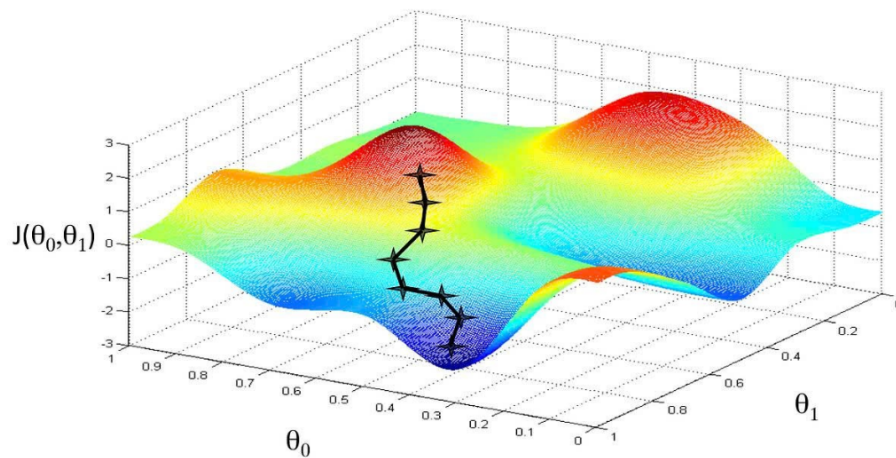
$$\delta^{(l)} = (W^{(l+1)})^T \delta^{(l+1)} \odot \sigma'^{(l)}(z^{(l)}) \quad (2.3)$$

Trong đó $\odot$ là phép nhân element-wise và σ' là đạo hàm của hàm kích hoạt. Sau khi tính được gradient, các tham số được cập nhật theo thuật toán tối ưu như Stochastic Gradient Descent (SGD):

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial W^{(l)}} \quad (2.4)$$

với η là tốc độ học (learning rate).

Ý nghĩa trực quan của Gradient Descent



Hình 2.3: Bề mặt hàm mất mát $J(\theta_0, \theta_1)$ và quỹ đạo của Gradient Descent (đường đen) tìm điểm cực tiểu.

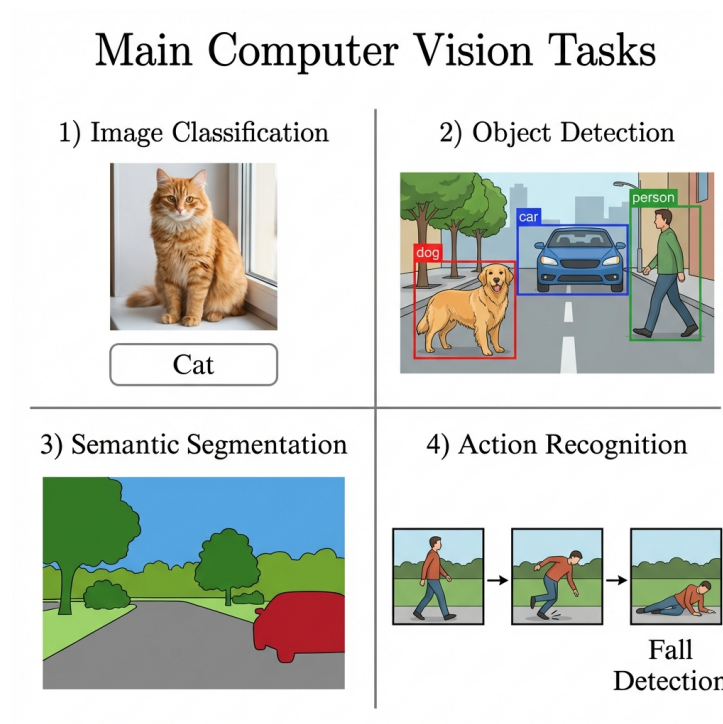
Công thức cập nhật trọng số có ý nghĩa hình học trực quan:

- **Bề mặt Loss:** Hàm mất mát $\mathcal{L}$ tạo thành một “bề mặt” trong không gian nhiều chiều, với mỗi điểm tương ứng với một tập giá trị trọng số W . Mục tiêu huấn luyện là tìm điểm thấp nhất trên bề mặt này (điểm cực tiểu).
- **Gradient:** Vector gradient $\nabla_W \mathcal{L} = \frac{\partial \mathcal{L}}{\partial W}$ chỉ hướng tăng nhanh nhất của hàm mất mát. Do đó, hướng ngược lại ($-\nabla_W \mathcal{L}$) chính là hướng giảm nhanh nhất.
- **Bước cập nhật:** Mỗi lần cập nhật, ta “đi” một bước theo hướng giảm nhanh nhất với độ lớn được kiểm soát bởi learning rate η :
 - η quá lớn: Bước đi quá dài, có thể “nhảy” qua điểm cực tiểu hoặc dao động không hội tụ
 - η quá nhỏ: Hội tụ rất chậm, tốn nhiều thời gian huấn luyện
 - η phù hợp: Hội tụ ổn định và nhanh đến điểm cực tiểu

- **Quỹ đạo tối ưu:** Qua nhiều iterations, các tham số được điều chỉnh dần để “lăn” xuống đáy của bề mặt loss, như hình minh họa với đường đen trên Hình 2.3.

2.2.4 Vai trò trong Thị giác Máy tính

Mạng nơ-ron đóng vai trò then chốt trong lĩnh vực thị giác máy tính (Computer Vision), cho phép máy tính “nhìn” và hiểu nội dung hình ảnh. Các ứng dụng tiêu biểu được minh họa trong Hình 2.4.



Hình 2.4: Bốn bài toán chính trong Thị giác Máy tính: Phân loại ảnh, Phát hiện đối tượng, Phân vùng ngữ nghĩa, và Nhận dạng hành động.

a) Phân loại ảnh (Image Classification)

Phân loại ảnh là bài toán cơ bản nhất trong thị giác máy tính, với mục tiêu gán một nhãn duy nhất cho toàn bộ ảnh đầu vào.

- **Đầu vào:** Một ảnh đơn lẻ (RGB hoặc grayscale)
- **Đầu ra:** Một nhãn lớp (class label) từ tập hợp các lớp định trước
- **Ví dụ:** Phân loại ảnh chứa chó, mèo, xe cộ; nhận diện chữ số viết tay (MNIST)
- **Ứng dụng:** Chẩn đoán y tế từ ảnh X-quang, nhận diện thực vật, kiểm tra chất lượng sản phẩm

b) Phát hiện đối tượng (Object Detection)

Phát hiện đối tượng mở rộng từ phân loại ảnh bằng cách không chỉ nhận diện loại đối tượng mà còn xác định vị trí của chúng trong ảnh.

- **Đầu vào:** Một ảnh có thể chứa nhiều đối tượng
- **Đầu ra:** Danh sách các bounding box (khung giới hạn) kèm nhãn lớp và điểm tin cậy (confidence score)
- **Các mô hình tiêu biểu:** YOLO (You Only Look Once), Faster R-CNN, SSD, RetinaNet
- **Ứng dụng:** Xe tự hành (phát hiện người đi bộ, biển báo), giám sát an ninh, đếm đối tượng

c) Phân vùng ngữ nghĩa (Semantic Segmentation)

Phân vùng ngữ nghĩa đưa việc hiểu ảnh lên mức pixel, gán nhãn lớp cho từng pixel trong ảnh.

- **Đầu vào:** Một ảnh có kích thước $H \times W$
- **Đầu ra:** Ma trận nhãn $H \times W$, mỗi pixel được gán một lớp
- **Các mô hình tiêu biểu:** U-Net, FCN (Fully Convolutional Networks), DeepLab, SegFormer

- **Ứng dụng:** Phân tích ảnh y tế (phân vùng khối u), xe tự hành (phân biệt đường, vỉa hè, xe cộ), chỉnh sửa ảnh

d) Nhận dạng hành động (Action Recognition)

Nhận dạng hành động mở rộng thị giác máy tính sang miền thời gian, phân tích chuỗi các frame video để xác định hành vi của con người hoặc đối tượng.

- **Đầu vào:** Video hoặc chuỗi các frame liên tiếp
- **Đầu ra:** Nhãn hành động (ví dụ: đi bộ, chạy, ngã, vẫy tay)
- **Các phương pháp:**
 - 3D CNN: Mở rộng CNN sang chiều thời gian (ví dụ: I3D, C3D)
 - Two-stream networks: Kết hợp RGB và optical flow
 - Transformer-based: Video Transformer, TimeSformer
- **Ứng dụng:** Phát hiện hành vi bất thường, nhận dạng cử chỉ, phân tích thể thao

e) Phát hiện té ngã (Fall Detection)

Đặc biệt quan trọng trong đề tài này, phát hiện té ngã là bài toán kết hợp nhiều kỹ thuật:

- **Object Detection:** Phát hiện và định vị người trong khung hình
- **Pose Estimation:** Ước lượng tư thế cơ thể (skeleton detection)
- **Action Recognition:** Phân tích chuỗi tư thế theo thời gian
- **Đặc trưng phân biệt:**
 - Góc nghiêng của cơ thể so với mặt đất
 - Tốc độ thay đổi vị trí (velocity)
 - Tỷ lệ chiều cao/chiều rộng của bounding box
 - Vị trí tương đối của các khớp (joints)

Mạng nơ-ron học được các đặc trưng phức tạp từ tư thế cơ thể, chuyển động, và ngữ cảnh không gian để phân biệt giữa các hành vi bình thường (ngồi xuống, cúi người) và trạng thái ngã thực sự.

2.3 Mạng Perceptron Đa tầng (MLP)

2.3.1 Định nghĩa MLP

Multilayer Perceptron (MLP) là một trong những kiến trúc mạng nơ-ron cơ bản nhất, thuộc lớp mạng truyền thẳng (feedforward neural network). MLP được cấu thành từ nhiều lớp neuron xếp chồng, trong đó tín hiệu chỉ truyền theo một chiều từ đầu vào đến đầu ra mà không có vòng lặp phản hồi.

MLP khác biệt với Perceptron đơn lớp ở chỗ có thêm một hoặc nhiều lớp ẩn (hidden layers) giữa lớp đầu vào và lớp đầu ra. Sự hiện diện của các lớp ẩn kết hợp với hàm kích hoạt phi tuyến cho phép MLP học được các hàm ánh xạ phức tạp, vượt qua giới hạn của các mô hình tuyến tính.

2.3.2 Cấu trúc nhiều lớp

Kiến trúc MLP tiêu biểu bao gồm ba loại lớp:

- **Lớp đầu vào (Input Layer):** Nhận dữ liệu thô và truyền đến các lớp tiếp theo. Số neuron bằng số chiều của vector đặc trưng đầu vào. Lớp này không thực hiện bất kỳ phép tính nào.
- **Lớp ẩn (Hidden Layers):** Một hoặc nhiều lớp thực hiện các phép biến đổi phi tuyến trên dữ liệu. Mỗi neuron trong lớp ẩn kết nối đầy đủ (fully connected) với tất cả neuron của lớp trước đó. Số lượng lớp ẩn và số neuron mỗi lớp là siêu tham số (hyperparameter) cần được lựa chọn phù hợp với bài toán.
- **Lớp đầu ra (Output Layer):** Tạo ra dự đoán cuối cùng của mô hình. Số neuron phụ thuộc vào bài toán: một neuron cho hồi quy, C neuron cho phân loại C lớp kết hợp với hàm Softmax.

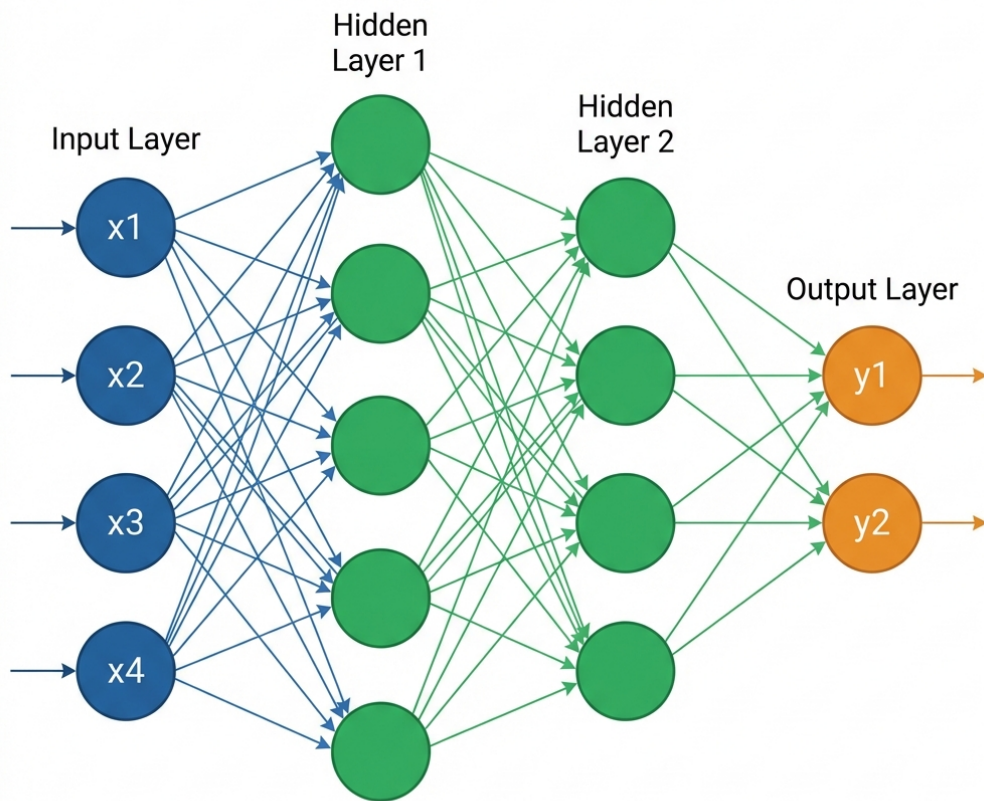
Ví dụ, một MLP với cấu trúc [784, 256, 128, 10] có 784 neuron đầu vào (phù hợp với ảnh MNIST 28×28 pixel), hai lớp ẩn với 256 và 128 neuron, và 10 neuron đầu ra cho 10 lớp chữ số.

2.3.3 Ưu điểm, hạn chế của MLP và giới hạn trong xử lý ảnh

So sánh ưu điểm và hạn chế của MLP

Hạn chế của MLP trong xử lý ảnh và video

Multilayer Perceptron (MLP) Architecture



Hình 2.5: Kiến trúc mạng Perceptron Đa tầng (MLP) với lớp đầu vào, hai lớp ẩn và lớp đầu ra. Mỗi neuron trong một lớp kết nối đầy đủ với tất cả neuron của lớp liền kề.

Bảng 2.1: Ưu điểm và hạn chế của mạng MLP

Ưu điểm	Hạn chế
Có khả năng xấp xỉ mọi hàm liên tục (Universal Approximation Theorem)	Số lượng tham số tăng nhanh theo kích thước đầu vào do kết nối đầy đủ
Cài đặt đơn giản, dễ hiểu và triển khai	Không khai thác được cấu trúc không gian của dữ liệu ảnh
Phù hợp với dữ liệu dạng bảng (tabular data) và các bài toán với đặc trưng được thiết kế sẵn	Nhạy cảm với nhiễu và biến đổi (dịch chuyển, xoay, co giãn) trong ảnh
Huấn luyện ổn định với các phương pháp tối ưu chuẩn	Dễ bị overfitting khi dữ liệu huấn luyện hạn chế

Mặc dù MLP có thể được áp dụng cho dữ liệu ảnh bằng cách “làm phẳng” (flatten) ma trận pixel thành vector một chiều, cách tiếp cận này có nhiều nhược điểm nghiêm

trọng:

1. **Mất thông tin không gian:** Khi chuyển đổi ảnh 2D thành vector 1D, quan hệ không gian giữa các pixel lân cận bị phá vỡ. Ví dụ, với ảnh $224 \times 224 \times 3$, MLP xem 150,528 giá trị pixel như các đặc trưng độc lập, mất đi khái niệm về “vùng”, “biên”, hay “hình dạng”.
2. **Số tham số khổng lồ:** Với ảnh $224 \times 224 \times 3$ và lớp ẩn đầu tiên có 1000 neuron, riêng lớp này đã cần $224 \times 224 \times 3 \times 1000 \approx 150$ triệu tham số, gây khó khăn cho việc huấn luyện và dễ dẫn đến overfitting.
3. **Không bất biến với phép biến đổi:** MLP không có tính bất biến tịnh tiến (translation invariance). Cùng một đối tượng xuất hiện ở vị trí khác trong ảnh sẽ tạo ra vector đầu vào hoàn toàn khác, khiến mô hình khó tổng quát hóa.
4. **Không thể xử lý video hiệu quả:** Video là chuỗi các khung hình, đòi hỏi xử lý thông tin thời gian. MLP không có cơ chế tự nhiên để mô hình hóa sự phụ thuộc theo thời gian.

Những hạn chế này thúc đẩy sự phát triển của mạng nơ-ron tích chập (CNN) - kiến trúc được thiết kế đặc biệt cho dữ liệu có cấu trúc không gian như ảnh và video.

2.4 Mạng Nơ-ron Tích chập (CNN)

2.4.1 Động cơ ra đời của CNN

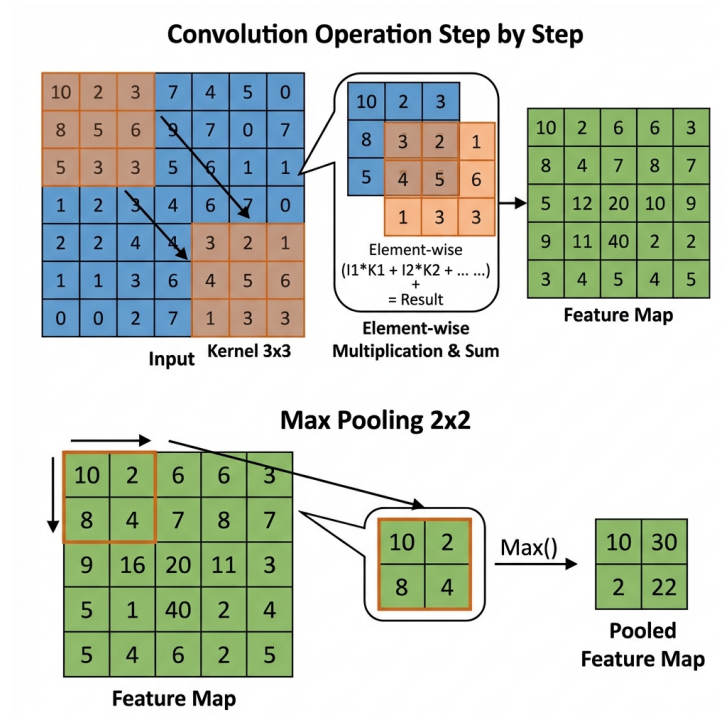
Như đã phân tích, MLP không phù hợp cho xử lý ảnh do không khai thác được cấu trúc không gian cục bộ. Trong khi đó, thị giác sinh học cho thấy não bộ xử lý thông tin thị giác theo cách phân cấp: các vùng thị giác sơ cấp phát hiện các đặc trưng đơn giản như cạnh và góc, sau đó các vùng cao hơn tổng hợp thành các khái niệm phức tạp hơn.

Năm 1998, Yann LeCun và cộng sự đề xuất kiến trúc LeNet-5 cho bài toán nhận dạng chữ số viết tay, đánh dấu sự ra đời của CNN hiện đại. CNN được thiết kế với ba ý tưởng cốt lõi:

- **Kết nối cục bộ (Local Connectivity):** Mỗi neuron chỉ kết nối với một vùng nhỏ của đầu vào thay vì toàn bộ, phù hợp với tính cục bộ của đặc trưng ảnh.
- **Chia sẻ tham số (Weight Sharing):** Cùng một bộ lọc được áp dụng trên toàn bộ ảnh, giảm đáng kể số tham số và tạo tính bất biến tịnh tiến.

- **Pooling:** Giảm kích thước không gian, tăng tính bất biến với biến đổi nhỏ và mở rộng trường nhìn (receptive field).

2.4.2 Các thành phần chính của CNN



Hình 2.6: Minh họa phép tích chập (Convolution) và Max Pooling trong CNN: bộ lọc 3×3 trượt qua ảnh đầu vào tạo ra feature map, sau đó Max Pooling 2×2 giảm kích thước không gian.

a) Lớp Tích chập (Convolution Layer)

Lớp tích chập là thành phần cốt lõi của CNN, thực hiện phép tích chập giữa đầu vào và các bộ lọc (filter/kernel) có thể học được. Mỗi bộ lọc trượt qua toàn bộ ảnh đầu vào, tính tích vô hướng tại mỗi vị trí để tạo ra bản đồ đặc trưng (feature map).

Với ảnh đầu vào I có kích thước $H \times W \times C$ và bộ lọc K có kích thước $k \times k \times C$, phép tích chập được định nghĩa:

$$(I * K)[i, j] = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} \sum_{c=0}^{C-1} I[i + m, j + n, c] \cdot K[m, n, c] + b \quad (2.5)$$

Các siêu tham số quan trọng của lớp tích chập:

- **Kích thước bộ lọc:** Thường là 3×3 hoặc 5×5
- **Bước nhảy (Stride):** Khoảng cách di chuyển của bộ lọc, stride lớn hơn làm giảm kích thước đầu ra
- **Padding:** Thêm viền 0 xung quanh ảnh để kiểm soát kích thước đầu ra

- **Số bộ lọc:** Quyết định số kênh của feature map đầu ra

b) Lớp Pooling

Lớp pooling thực hiện phép lấy mẫu xuống (downsampling) để giảm kích thước không gian của feature map, từ đó giảm số tham số và chi phí tính toán, đồng thời tăng tính bất biến với các biến đổi nhỏ.

Hai loại pooling phổ biến:

- **Max Pooling:** Lấy giá trị lớn nhất trong mỗi vùng, giữ lại đặc trưng nổi bật nhất
- **Average Pooling:** Lấy giá trị trung bình, cho kết quả mượt mà hơn

Ví dụ, Max Pooling với cửa sổ 2×2 và stride 2 sẽ giảm kích thước feature map đi một nửa theo mỗi chiều.

c) Lớp Kết nối Đầy đủ (Fully Connected Layer)

Sau các lớp tích chập và pooling, feature map được “làm phẳng” thành vector và đưa qua các lớp fully connected để thực hiện phân loại hoặc hồi quy. Các lớp này hoạt động như MLP, tổng hợp các đặc trưng đã trích xuất để đưa ra dự đoán cuối cùng.

2.4.3 Ưu điểm của CNN trong xử lý ảnh và video

- **Học đặc trưng tự động:** CNN tự động trích xuất các đặc trưng phân cấp từ thấp đến cao (cạnh $\rightarrow$ texture $\rightarrow$ bộ phận $\rightarrow$ đối tượng) mà không cần thiết kế thủ công.
- **Hiệu quả tham số:** Nhờ chia sẻ trọng số, CNN có ít tham số hơn nhiều so với MLP tương đương.
- **Bất biến tịnh tiến:** Cùng một bộ lọc áp dụng trên toàn ảnh giúp phát hiện đặc trưng bất kể vị trí.
- **Xử lý video tự nhiên:** CNN có thể mở rộng sang 3D-CNN để xử lý cả chiều thời gian trong video.

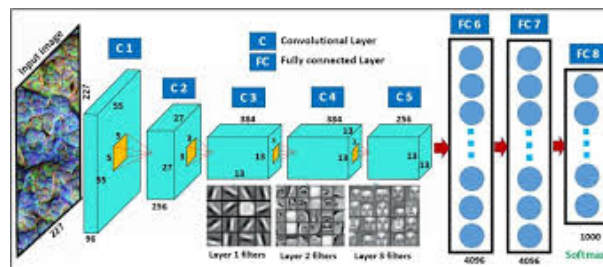
2.4.4 Các kiến trúc CNN tiêu biểu

a) AlexNet (2012)

AlexNet được Alex Krizhevsky và cộng sự phát triển, giành chiến thắng tại cuộc thi ImageNet 2012 với cách biệt lớn, đánh dấu sự bùng nổ của học sâu trong thị giác máy tính.

Đặc điểm chính của AlexNet:

- 5 lớp tích chập và 3 lớp fully connected, tổng cộng khoảng 60 triệu tham số
- Sử dụng hàm kích hoạt ReLU thay cho Sigmoid/Tanh, giúp huấn luyện nhanh hơn
- Áp dụng Dropout để giảm overfitting
- Huấn luyện trên 2 GPU song song lần đầu tiên



Hình 2.7: AlexNet

b) VGGNet (2014)

VGGNet do nhóm Visual Geometry Group tại Oxford phát triển, nổi bật với triết lý thiết kế đơn giản và đồng nhất: sử dụng các bộ lọc 3×3 xuyên suốt.

Kiến trúc VGG-16 bao gồm 13 lớp tích chập và 3 lớp fully connected. Ý tưởng chính là thay vì dùng bộ lọc lớn 7×7 , có thể xếp chồng nhiều bộ lọc 3×3 để đạt cùng receptive field nhưng với ít tham số hơn và thêm các lớp phi tuyến.



Hình 2.8: VGG

c) ResNet (2015)

ResNet (Residual Network) do nhóm Microsoft Research phát triển, giải quyết vấn đề nghiêm trọng của các mạng rất sâu: hiện tượng suy giảm gradient (vanishing gradient) và sự suy giảm độ chính xác.

Vấn đề: Khi mạng sâu hơn, lý thuyết cho rằng độ chính xác phải tốt lên hoặc ít nhất không kém đi (vì mạng sâu có thể “học” thành mạng nông). Tuy nhiên thực tế, mạng quá sâu có độ chính xác giảm cả trên tập huấn luyện lẫn kiểm tra.

Giải pháp - Residual Connection: Thay vì học ánh xạ $H(x)$ trực tiếp, ResNet học “phần dư” $F(x) = H(x) - x$ và đầu ra là:

$$y = F(x, \{W_i\}) + x \quad (2.6)$$

Kết nối tắt (skip connection) $+x$ cho phép gradient lan truyền trực tiếp qua các lớp, giải quyết vanishing gradient. Nếu $F(x)$ không cần thiết, mạng có thể dễ dàng học $F(x) \approx 0$ để trở thành hàm đồng nhất.

ResNet cho phép huấn luyện mạng với hàng trăm thậm chí hàng nghìn lớp (ResNet-152, ResNet-1001), đạt kết quả vượt trội trên nhiều benchmark và trở thành nền tảng cho nhiều kiến trúc hiện đại.

2.5 Giới thiệu YOLO (You Only Look Once)

2.5.1 Bài toán Object Detection

Object Detection (Phát hiện đối tượng) là một trong những bài toán cơ bản và quan trọng nhất trong thị giác máy tính [13]. Khác với Image Classification chỉ xác định loại đối tượng chính trong ảnh, Object Detection yêu cầu đồng thời:

- **Định vị (Localization):** Xác định vị trí của đối tượng trong ảnh thông qua bounding box (hộp giới hạn)
- **Phân loại (Classification):** Xác định loại của mỗi đối tượng được phát hiện

Trong bài toán phát hiện ngã, Object Detection đóng vai trò then chốt: mô hình cần xác định vị trí con người trong khung hình và phân loại trạng thái của họ (đang đứng, đi lại, hay đã ngã).

2.5.2 Lịch sử phát triển Object Detection

Theo survey “Object Detection in 20 Years” của Zou et al. [13], lĩnh vực Object Detection đã trải qua sự phát triển đáng kể trong hơn hai thập kỷ:

a) Giai đoạn truyền thống (1990s-2012)

- **Viola-Jones Detector (2001):** Detector đầu tiên đạt tốc độ real-time cho face detection, sử dụng Haar-like features và AdaBoost

- **HOG Detector (2005)**: Histogram of Oriented Gradients cho pedestrian detection
- **DPM - Deformable Parts Model (2010)**: Mô hình biến dạng các phần, đạt SOTA trên PASCAL VOC nhiều năm

b) Giai đoạn Deep Learning (2012-nay)

- **AlexNet (2012)**: Bước ngoặt của deep learning trong computer vision
- **R-CNN (2014)**: Kết hợp region proposals với CNN features
- **Fast R-CNN, Faster R-CNN (2015)**: Cải tiến tốc độ với RoI pooling và Region Proposal Network
- **YOLO v1 (2016)**: Đột phá với one-stage detection, real-time
- **SSD (2016)**: Single Shot MultiBox Detector với multi-scale features
- **RetinaNet (2017)**: Giới thiệu Focal Loss để giải quyết class imbalance
- **YOLO v3-v11 (2018-2024)**: Liên tục cải tiến về accuracy và speed

c) Xu hướng hiện tại

Theo [13], các xu hướng chính trong Object Detection bao gồm:

- Anchor-free detectors (loại bỏ anchor boxes)
- Transformer-based detectors (DETR, Swin Transformer)
- NAS-based architecture search (tự động tìm kiếm kiến trúc)
- Real-time detection trên edge devices

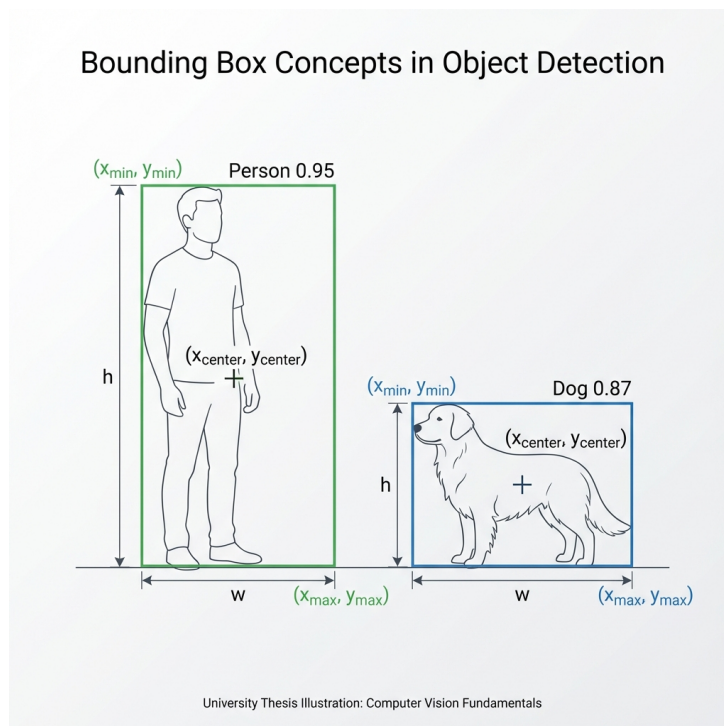
2.5.3 Khái niệm Bounding Box

Bounding Box (hộp giới hạn) là hình chữ nhật bao quanh đối tượng trong ảnh, được sử dụng để xác định vị trí và kích thước của đối tượng.

Có hai cách biểu diễn bounding box phổ biến:

a) Format $(x_{min}, y_{min}, x_{max}, y_{max})$ - PASCAL VOC format

- (x_{min}, y_{min}) : Tọa độ góc trên bên trái của box



Hình 2.9: Ví dụ minh họa Bounding Box trong Object Detection.

- (x_{max}, y_{max}) : Tọa độ góc dưới bên phải của box
- Đơn vị: pixel tuyệt đối

b) Format $(x_{center}, y_{center}, w, h)$ - YOLO format

- (x_{center}, y_{center}) : Tọa độ tâm của box
- w : Chiều rộng của box
- h : Chiều cao của box
- Đơn vị: normalized (chia cho kích thước ảnh, giá trị từ 0 đến 1)

c) Công thức chuyển đổi

Từ PASCAL VOC sang YOLO (với ảnh kích thước $W \times H$):

$$x_{center} = \frac{x_{min} + x_{max}}{2W}, \quad y_{center} = \frac{y_{min} + y_{max}}{2H} \quad (2.7)$$

$$w = \frac{x_{max} - x_{min}}{W}, \quad h = \frac{y_{max} - y_{min}}{H} \quad (2.8)$$

2.5.4 Intersection over Union (IoU)

IoU là metric đo độ chồng lấp giữa hai bounding box, được sử dụng rộng rãi trong đánh giá object detection:

$$IoU = \frac{\text{Area of Intersection}}{\text{Area of Union}} = \frac{|A \cap B|}{|A \cup B|} \quad (2.9)$$

Trong đó:

- A : Bounding box dự đoán (predicted)
- B : Bounding box ground truth (thực tế)
- $IoU = 1$: Hai box trùng khớp hoàn toàn
- $IoU = 0$: Hai box không có phần giao nhau

Ngưỡng IoU thường được sử dụng để xác định dự đoán đúng:

- $IoU \geq 0.5$: Ngưỡng phổ biến (PASCAL VOC, COCO mAP@0.5)
- $IoU \geq 0.75$: Ngưỡng nghiêm ngặt (COCO mAP@0.75)
- IoU từ 0.5 đến 0.95: Đánh giá đa ngưỡng (COCO mAP@[.5:.95])

2.5.5 Các khái niệm True Positive, False Positive, False Negative

Để đánh giá kết quả detection, cần phân loại các dự đoán:

- **True Positive (TP)**: Dự đoán đúng - có bounding box dự đoán khớp với ground truth ($IoU \geq \text{threshold}$) và đúng class
- **False Positive (FP)**: Dự đoán sai - bounding box dự đoán không khớp với bất kỳ ground truth nào, hoặc sai class
- **False Negative (FN)**: Bỏ sót - ground truth không được detect bởi bất kỳ dự đoán nào

Từ đó tính được:

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{\text{Số dự đoán đúng}}{\text{Tổng số dự đoán}} \quad (2.10)$$

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{\text{Số dự đoán đúng}}{\text{Tổng số ground truth}} \quad (2.11)$$

2.5.6 Đánh giá hiệu suất: Mean Average Precision (mAP)

Mean Average Precision (mAP) là metric chuẩn để đánh giá hiệu suất của mô hình object detection, tổng hợp cả precision và recall.

a) Precision-Recall Curve

Với mỗi class, sắp xếp các dự đoán theo confidence score giảm dần. Tại mỗi điểm, tính precision và recall tích lũy, từ đó vẽ được đường cong Precision-Recall.

b) Average Precision (AP)

AP là diện tích dưới đường cong Precision-Recall:

$$AP = \int_0^1 p(r) dr \quad (2.12)$$

Trong thực tế, sử dụng phép nội suy (interpolation):

$$AP = \sum_{i=1}^n (r_i - r_{i-1}) \cdot p_{interp}(r_i) \quad (2.13)$$

với $p_{interp}(r) = \max_{r' \geq r} p(r')$ là precision nội suy tại recall r .

c) Mean Average Precision (mAP)

mAP là trung bình của AP trên tất cả các class:

$$mAP = \frac{1}{N} \sum_{i=1}^N AP_i \quad (2.14)$$

với N là số class trong dataset.

d) Các biến thể mAP

- **mAP@0.5 (mAP50):** AP tính với ngưỡng IoU = 0.5, được sử dụng trong PASCAL VOC
- **mAP@0.75 (mAP75):** AP với ngưỡng IoU = 0.75, nghiêm ngặt hơn
- **mAP@[.5:.95]:** Trung bình AP với IoU từ 0.5 đến 0.95 (bước 0.05), metric chính

của COCO

- **mAP Small/Medium/Large:** mAP theo kích thước đối tượng

e) Ý nghĩa của mAP

- mAP cao $\rightarrow$ Mô hình phát hiện đúng nhiều đối tượng và định vị chính xác
- mAP50 cao nhưng mAP75 thấp $\rightarrow$ Phát hiện được nhưng định vị chưa chính xác
- Precision cao, Recall thấp $\rightarrow$ Mô hình thận trọng, ít false positive nhưng bỏ sót nhiều
- Precision thấp, Recall cao $\rightarrow$ Mô hình phát hiện nhiều nhưng có nhiều false positive

2.5.7 Ý tưởng YOLO: Phát hiện trong một lần suy luận

YOLO (You Only Look Once) được Joseph Redmon, Santosh Divvala, Ross Girshick và Ali Farhadi đề xuất năm 2016 trong bài báo “You Only Look Once: Unified, Real-Time Object Detection” [1]. Đây là bước đột phá mang đến cách tiếp cận hoàn toàn mới cho Object Detection.

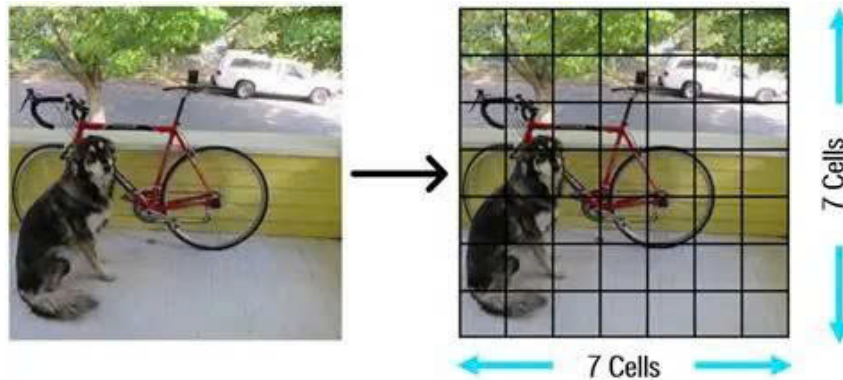
Ý tưởng cách mạng của YOLO (theo [1]):

“We reframe object detection as a single regression problem, straight from image pixels to bounding box coordinates and class probabilities. Using our system, you only look once (YOLO) at an image to predict what objects are present and where they are.”

Thay vì coi detection như bài toán phân loại (classifier), YOLO đặt lại vấn đề như bài toán hồi quy về tọa độ bounding box và xác suất class, tất cả trong một lần forward pass duy nhất qua mạng neural.

Nguyên lý hoạt động của YOLO:

1. **Chia lưới:** Ảnh đầu vào được chia thành lưới $S \times S$ ô (grid cells). Trong paper gốc, $S = 7$.
2. **Dự đoán bounding boxes:** Mỗi ô dự đoán B bounding boxes (mặc định $B = 2$) cùng với điểm confidence.
3. **Dự đoán class probabilities:** Mỗi ô dự đoán C xác suất class có điều kiện $Pr(Class_i|Object)$.



Hình 2.10: Minh họa chia lưới 7×7 trong YOLO: mỗi ô chịu trách nhiệm phát hiện đối tượng có tâm nằm trong ô đó.

4. **Tensor đầu ra:** Kích thước $S \times S \times (B \times 5 + C)$. Với PASCAL VOC ($C = 20$), tensor có kích thước $7 \times 7 \times 30$.
5. **Post-processing:** Non-Maximum Suppression (NMS) loại bỏ các box trùng lặp.

Đầu ra của mỗi bounding box bao gồm 5 giá trị:

- (x, y) : Tọa độ tâm của box tương đối với ô lưới (normalized 0-1 trong ô)
- (w, h) : Chiều rộng và chiều cao của box tương đối với toàn ảnh (normalized 0-1)
- $confidence = Pr(Object) \times IOU_{pred}^{truth}$: Độ tin cậy rằng box chứa đối tượng VÀ độ chính xác của box so với ground truth

Hiệu suất của YOLO theo bài báo gốc:

- Base YOLO: 45 FPS (frames per second) trên GPU
- Fast YOLO: 155 FPS - nhanh hơn gấp đôi các detector real-time khác
- mAP trên PASCAL VOC 2007: 63.4% (base), 52.7% (fast)

2.5.8 So sánh YOLO với phương pháp truyền thống

Ưu điểm của YOLO:

- Tốc độ: Xử lý real-time với hàng chục đến hàng trăm FPS
- Đơn giản: Một mạng duy nhất, dễ huấn luyện end-to-end

Bảng 2.2: So sánh YOLO với các phương pháp Object Detection khác

Phương pháp	Tốc độ (FPS)	mAP	Thời gian thực
R-CNN	0.02	53.7	Không
Fast R-CNN	0.5	70.0	Không
Faster R-CNN	7-17	73.2	Khó
YOLO v1	45	63.4	Có
YOLOv3	30-50	57.9	Có
YOLOv5	100+	68.9	Có

- Nhìn toàn cục: Xem xét toàn bộ ảnh khi dự đoán, hiểu được ngữ cảnh tốt hơn
- Tổng quát hóa tốt: Ít false positive trên background so với các phương pháp dựa trên sliding window

Hạn chế:

- Khó phát hiện các đối tượng nhỏ hoặc xếp chồng nhau
- Độ chính xác định vị có thể kém hơn phương pháp hai giai đoạn ở một số trường hợp

2.6 Lý do lựa chọn YOLOv11 cho bài toán Fall Detection

2.6.1 Giới thiệu YOLOv11

YOLOv11 là phiên bản mới nhất trong họ YOLO, được phân tích chi tiết trong nghiên cứu “YOLOv11: An Overview of the Key Architectural Enhancements” của Rahima Khanam và Muhammad Hussain [11]. Phiên bản này được phát triển bởi Ultralytics (2024), kế thừa và cải tiến từ YOLOv5 và YOLOv8.

Theo [11], các cải tiến kiến trúc chính của YOLOv11 bao gồm:

- **C3k2 Block (Cross Stage Partial with kernel size 2):** Module convolution mới thay thế C3 trong YOLOv8, sử dụng kernel size nhỏ hơn để tăng tốc độ inference trong khi vẫn giữ hiệu suất feature extraction.
- **SPPF (Spatial Pyramid Pooling - Fast):** Module pooling nhanh hơn SPP truyền thống, cho phép trích xuất đặc trưng đa tỉ lệ hiệu quả.
- **C2PSA (Convolutional block with Parallel Spatial Attention):** Module attention mới giúp mô hình tập trung vào các vùng quan trọng trong ảnh, cải thiện khả năng phát hiện đối tượng trong bối cảnh phức tạp.

- **Neck với PAnet cải tiến:** Path Aggregation Network kết hợp đặc trưng từ nhiều mức (multi-scale feature fusion).
- **Head anchor-free:** Loại bỏ anchor boxes, sử dụng dự đoán trực tiếp tọa độ, giảm hyperparameter cần tinh chỉnh.

YOLOv11 hỗ trợ đa nhiệm vụ trong cùng một framework: Object Detection, Instance Segmentation, Pose Estimation, Oriented Object Detection (OBB), và Classification.

2.6.2 Các phiên bản mô hình YOLOv11

Theo [11], YOLOv11 cung cấp nhiều kích thước mô hình phù hợp với các nhu cầu khác nhau:

Model	Params (M)	mAP@50-95	Latency (ms)	Ứng dụng
YOLOv11n	2.6	39.5	1.55	Đề xuất - Edge/Mobile
YOLOv11s	9.4	47.0	2.46	Cân bằng tốc độ/accuracy
YOLOv11m	20.1	51.5	4.70	Desktop GPU
YOLOv11l	25.3	53.4	6.16	High accuracy
YOLOv11x	56.9	54.7	11.31	Maximum accuracy

Nguồn: Ultralytics YOLO11 Documentation, 2024

2.7 Theo dõi và Nhận diện Người (Person Tracking)

2.7.1 Giới thiệu Person Tracking

Person Tracking (Theo dõi người) là bài toán xác định và theo dõi liên tục vị trí của người trong video qua các khung hình liên tiếp. Khác với Object Detection chỉ phát hiện người trong từng frame độc lập, Person Tracking duy trì identity của từng người xuyên suốt video.

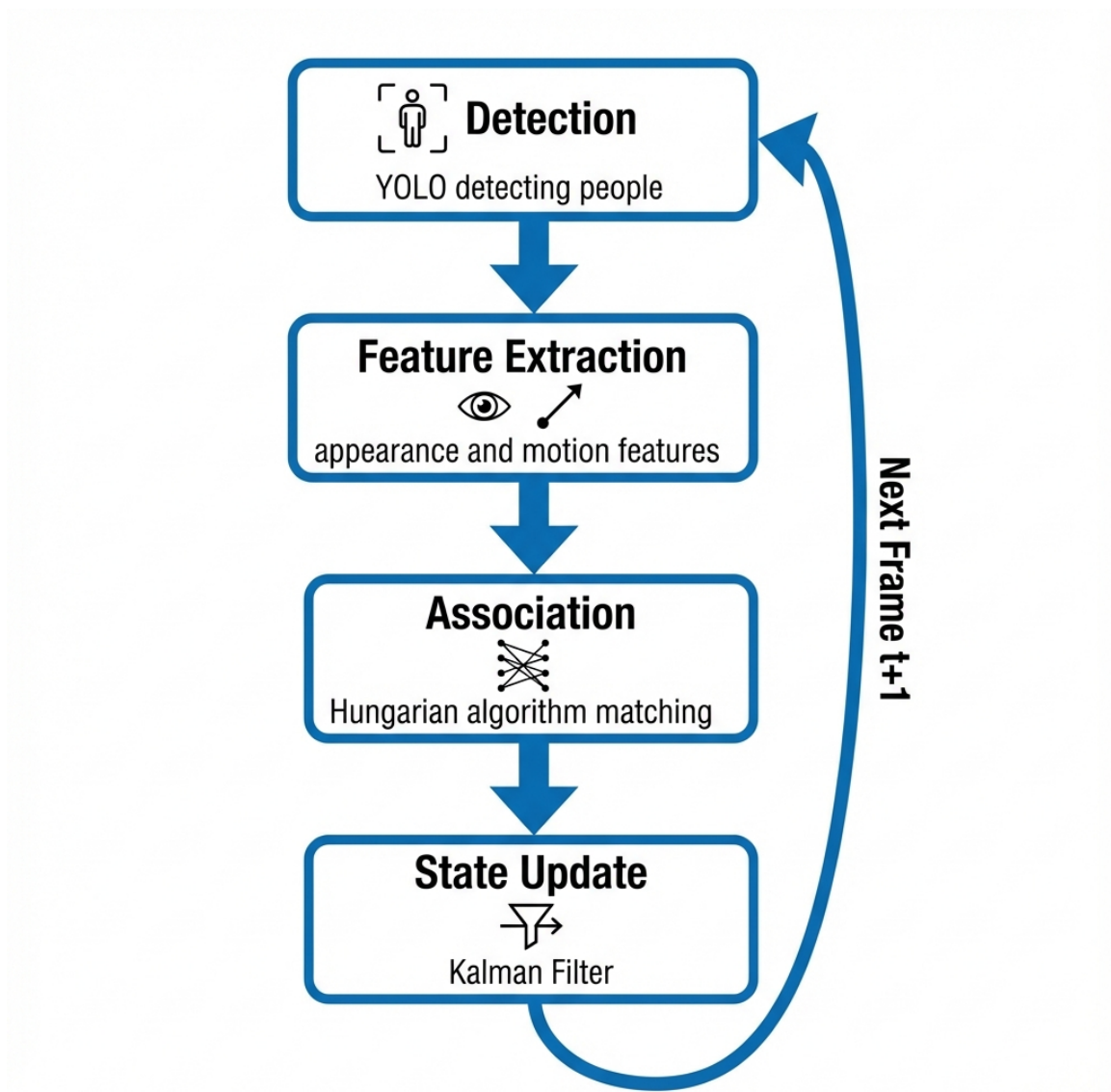
Yêu cầu của hệ thống Person Tracking:

- **Detection:** Phát hiện người trong mỗi frame
- **Association:** Liên kết các detection giữa các frame

- **ID Management:** Gán và duy trì ID duy nhất cho mỗi người
- **Re-identification:** Nhận diện lại người sau khi bị che khuất

2.7.2 Tracking-by-Detection Paradigm

Phương pháp Tracking-by-Detection là hướng tiếp cận phổ biến nhất, hoạt động theo pipeline 4 bước như minh họa trong Hình 2.11.



Hình 2.11: Flowchart pipeline Tracking-by-Detection: Detection → Feature Extraction → Association → State Update, lặp lại cho mỗi frame.

- **Detection:** Sử dụng detector (YOLO, Faster R-CNN) phát hiện người trong mỗi frame
- **Feature Extraction:** Trích xuất đặc trưng appearance và motion cho mỗi detection
- **Association:** So khớp detection với các track đang hoạt động (Hungarian Algorithm)
- **State Update:** Cập nhật trạng thái track bằng Kalman Filter (vị trí, velocity)

2.8 Giới thiệu Dataset Fall Detection – CauCaFall

2.8.1 Mục tiêu và đặc điểm của dataset CauCaFall

CauCaFall là bộ dữ liệu chuyên biệt cho bài toán phát hiện ngã, được xây dựng với mục tiêu cung cấp nguồn dữ liệu chất lượng cao cho nghiên cứu và phát triển các hệ thống giám sát an toàn trong môi trường sinh hoạt.

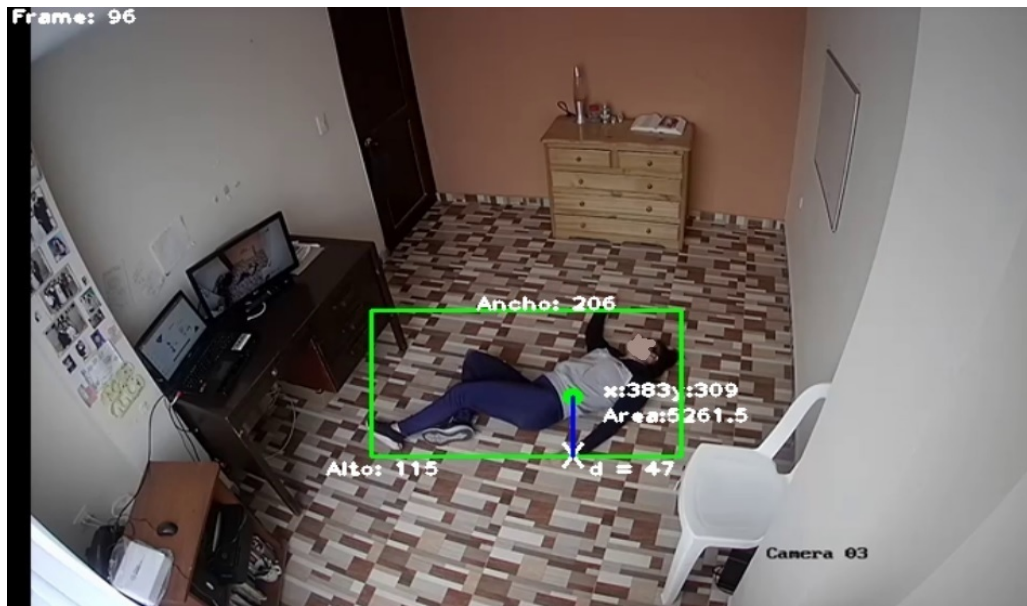
Đặc điểm nổi bật của dataset CauCaFall:

- **Môi trường trong nhà thực tế:** Dữ liệu được thu thập trong các không gian sống thực như phòng khách, phòng ngủ, hành lang, phản ánh đúng môi trường triển khai thực tế của hệ thống giám sát
- **Đa dạng góc quay:** Camera được đặt ở nhiều vị trí và góc độ khác nhau, mô phỏng các thiết lập giám sát phổ biến
- **Nhiều kịch bản ngã:** Bao gồm các tình huống ngã đa dạng từ trượt ngã, ngã do mất thăng bằng, đến ngã từ ghế/giường
- **Đa dạng đối tượng:** Nhiều người tham gia với độ tuổi, giới tính, và trang phục khác nhau

2.8.2 Dữ liệu video trong nhà

Dataset CauCaFall chủ yếu bao gồm các video được quay trong môi trường trong nhà với các đặc điểm kỹ thuật:

- Độ phân giải video đa dạng từ 720p đến 1080p
- Frame rate tiêu chuẩn 25-30 FPS



- Điều kiện ánh sáng đa dạng: ánh sáng tự nhiên ban ngày, đèn nhân tạo ban đêm
- Background bao gồm các đồ vật trong nhà phổ biến: bàn, ghế, giường, tủ

Video được cắt thành các clip ngắn, mỗi clip chứa một sự kiện cụ thể (ngã hoặc không ngã), thuận tiện cho việc huấn luyện và đánh giá mô hình.

2.8.3 Annotation ban đầu

Dataset CauCaFall gốc được gán nhãn với hai lớp cơ bản:

- **fall (1)**: Đánh dấu các khung hình hoặc video clip có người đang trong trạng thái ngã hoặc vừa ngã
- **no_fall (0)**: Đánh dấu các hoạt động bình thường như đi bộ, ngồi, đứng, nằm nghỉ ngơi (không phải do ngã)

Annotation được thực hiện ở mức bounding box cho từng frame, xác định vị trí của người trong ảnh kèm theo nhãn trạng thái.

2.8.4 Ưu điểm và hạn chế của dataset CauCaFall

Ưu điểm:

- Dữ liệu thực tế, phản ánh đúng môi trường ứng dụng
- Chất lượng video tốt, phù hợp cho các mô hình deep learning

- Có diversity về tình huống ngã và đối tượng
- Được gán nhãn cẩn thận ở mức bounding box

Hạn chế:

- Chỉ có hai lớp **fall/no_fall**, thiếu thông tin về các đối tượng khác trong cảnh
- Số lượng mẫu hạn chế so với các dataset lớn như COCO
- Không có annotation cho indoor objects (bàn, ghế, giường) - thông tin quan trọng cho việc hiểu ngữ cảnh
- Class imbalance: số mẫu no_fall thường nhiều hơn fall

2.8.5 Nghiên cứu liên quan: Dataset DiverseFALL10500

Nghiên cứu của Habib Khan và cộng sự (2024) “Visionary vigilance: Optimized YOLOV8 for fallen person detection with large-scale benchmark dataset” [12] đã đề xuất một dataset quy mô lớn cho bài toán Fall Detection:

DiverseFALL10500 Dataset:

- **Quy mô:** Khoảng 10,500 ảnh với annotations tương ứng
- **Đa dạng môi trường:** Indoor và outdoor scenes
- **Điều kiện ánh sáng:** Nhiều mức độ chiếu sáng khác nhau (illumination variations)
- **Đa dạng đối tượng:** Samples từ nhiều độ tuổi khác nhau
- **Góc quay đa dạng**
- **Mã nguồn mở:** Available tại <https://github.com/habib1402/Fall-Detection-DiverseFALL10500>

Theo [12], các dataset Fall Detection hiện có thường gặp hạn chế về:

- Thiếu đa dạng trong samples (môi trường đồng nhất)
- Hạn chế về biểu hiện thị giác của các nhóm tuổi khác nhau
- Thiếu coverage về điều kiện môi trường và ánh sáng
- Hạn chế về đa dạng trang phục

So sánh với CauCaFall:

Tiêu chí	CauCaFall	DiverseFALL10500
Số lượng samples	3,000 frames	10,500 images
Môi trường	Indoor only	Indoor + Outdoor
Điều kiện ánh sáng	Hạn chế	Đa dạng
Nhóm tuổi	Hạn chế	Đa dạng
Góc camera	Cố định	Đa dạng

Đồ án này sử dụng CauCaFall do tính sẵn có và phù hợp với môi trường trong nhà (indoor surveillance), đồng thời kết hợp với COCO để tăng cường đa dạng và annotation cho indoor objects.

2.9 Hệ thống AI Agent và Mô hình Ngôn ngữ Lớn (LLM)

2.9.1 Khái niệm AI Agent

AI Agent (Tác tử trí tuệ nhân tạo) là một hệ thống tự động có khả năng nhận thức môi trường, đưa ra quyết định, và thực hiện hành động để đạt được mục tiêu cụ thể. Khác với các chương trình truyền thống hoạt động theo luật cố định, AI Agent có khả năng:

- **Tự chủ (Autonomy):** Hoạt động độc lập mà không cần sự can thiệp liên tục của con người
- **Phản ứng (Reactivity):** Nhận thức và phản hồi kịp thời với thay đổi của môi trường
- **Chủ động (Proactivity):** Khởi xướng hành động để đạt mục tiêu, không chỉ phản ứng thụ động
- **Xã hội (Social ability):** Tương tác và giao tiếp với con người hoặc các agent khác

Trong hệ thống robot giám sát người cao tuổi, AI Agent đóng vai trò là “bộ não” thông minh, điều phối các thành phần khác nhau (camera, cảm biến, IoT) và giao tiếp tự nhiên với người dùng.

2.9.2 Mô hình Ngôn ngữ Lớn (Large Language Models)

Large Language Models (LLM) là các mô hình học sâu được huấn luyện trên lượng dữ liệu văn bản khổng lồ, có khả năng hiểu và sinh ngôn ngữ tự nhiên ở mức độ cao. LLM đã tạo ra bước đột phá trong AI conversational.

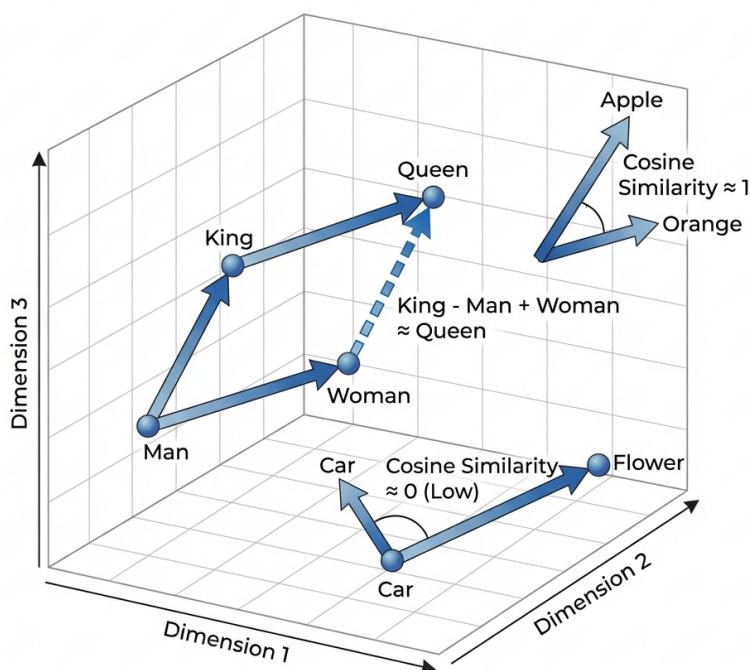
a) Kiến trúc Transformer

LLM hiện đại dựa trên kiến trúc Transformer được giới thiệu trong bài báo “Attention Is All You Need” (Vaswani et al., 2017). Transformer đã tạo ra một cuộc cách mạng trong xử lý ngôn ngữ tự nhiên bằng việc loại bỏ hoàn toàn các cấu trúc hồi quy (RNN, LSTM) và thay thế bằng cơ chế self-attention.

Lớp Embedding và Ánh xạ Không gian Đa chiều

Trước khi dữ liệu được đưa vào Transformer để huấn luyện, mỗi token (từ, ký tự, hoặc subword) cần phải đi qua lớp Embedding. Lớp này có chức năng ánh xạ dữ liệu từ không gian rời rạc (discrete space) sang không gian vector liên tục đa chiều (continuous high-dimensional space).

WORD EMBEDDING CONCEPT IN NLP



Hình 2.12: Minh họa không gian vector embedding và độ tương đồng cosine giữa các từ

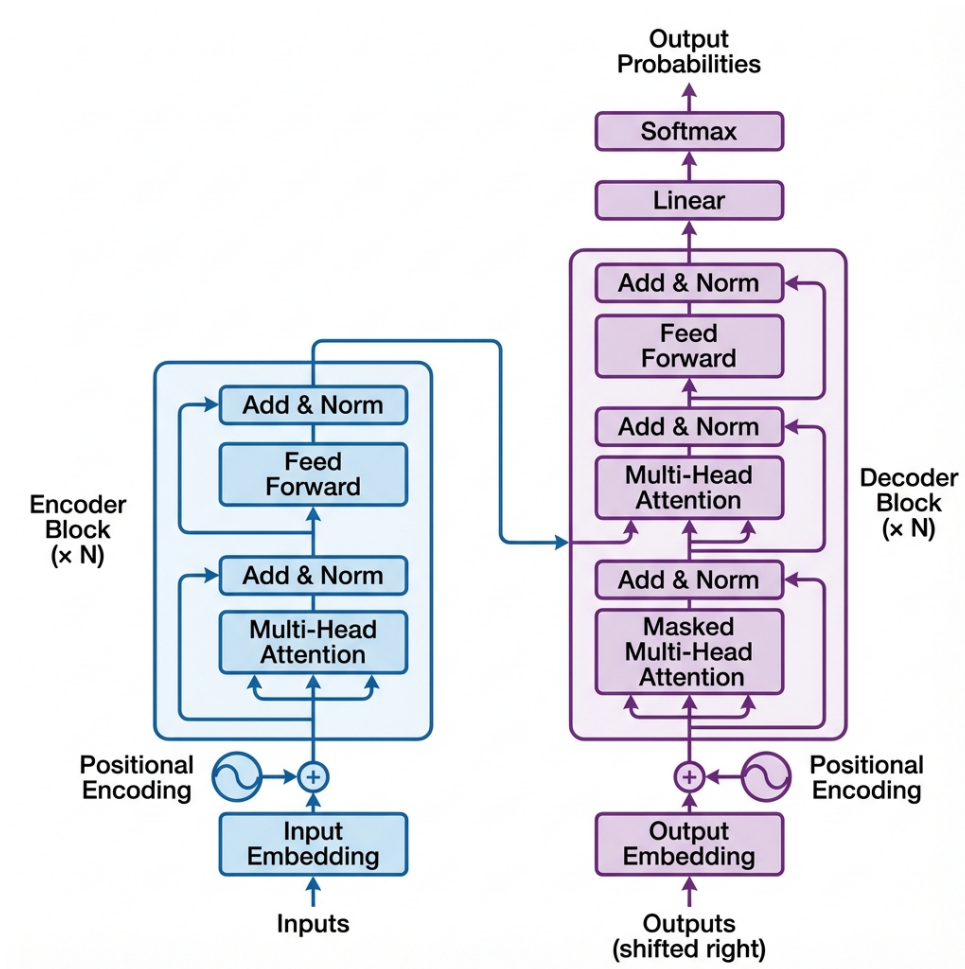
Trong không gian embedding đa chiều (thường từ 512 đến 4096 chiều), các từ có ngữ nghĩa tương đồng sẽ có vector gần nhau. Độ tương đồng giữa hai vector thường được đo bằng cosine similarity:

$$\text{cosine_similarity}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \cdot \|\mathbf{b}\|} = \frac{\sum_{i=1}^n a_i b_i}{\sqrt{\sum_{i=1}^n a_i^2} \cdot \sqrt{\sum_{i=1}^n b_i^2}} \quad (2.15)$$

Việc embedding giúp “làm dày” ngữ cảnh của mỗi data point, cho phép mô hình học được các mối quan hệ ngữ nghĩa phức tạp như:

- $\text{vector}(\text{“King”}) - \text{vector}(\text{“Man”}) + \text{vector}(\text{“Woman”}) \approx \text{vector}(\text{“Queen”})$
- Các từ đồng nghĩa có vector gần nhau trong không gian embedding
- Các từ trong cùng ngữ cảnh có cosine similarity cao

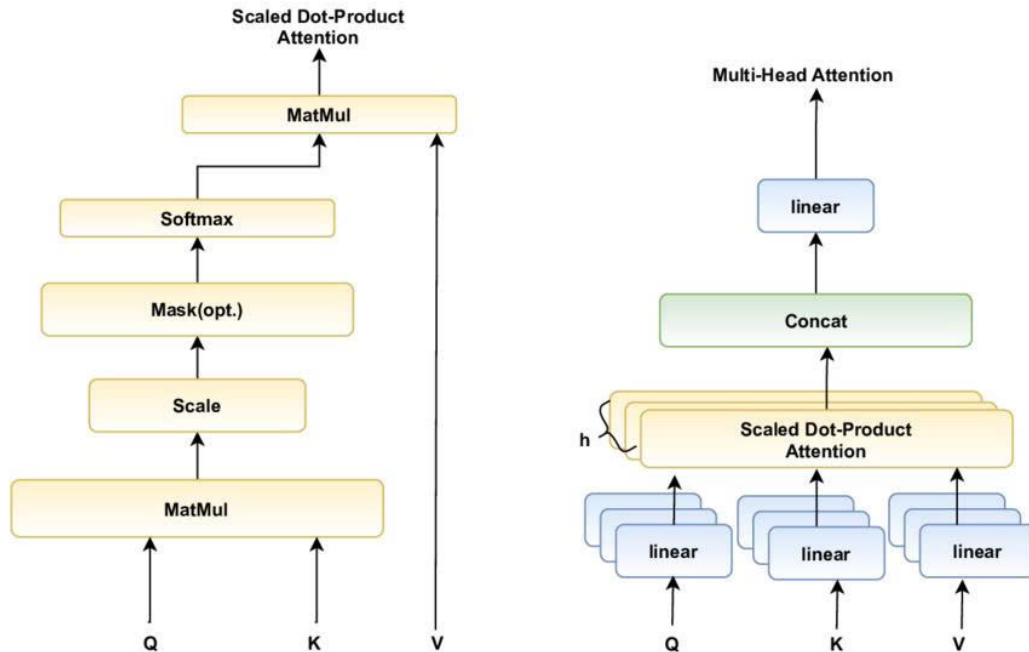
Cơ chế Self-Attention



Hình 2.13: Kiến trúc Transformer với các khối Encoder và Decoder

Self-Attention cho phép mỗi vị trí trong chuỗi đầu vào “chú ý” đến tất cả các vị trí khác, học được mối quan hệ giữa các từ bất kể khoảng cách. Cơ chế này hoạt động thông qua ba ma trận: Query (Q), Key (K), và Value (V):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (2.16)$$



Hình 2.14: Cơ chế Self-Attention: Query, Key, Value và Scaled Dot-Product Attention

Trong đó:

- Q (Query): Biểu diễn “câu hỏi” - token hiện tại muốn tìm thông tin gì
- K (Key): Biểu diễn “chìa khóa” - các token khác chứa thông tin gì
- V (Value): Biểu diễn “giá trị” - nội dung thực sự của các token
- d_k : Chiều của vector key, dùng để scale gradient ổn định

Multi-Head Attention mở rộng cơ chế này bằng cách chạy song song nhiều attention head, mỗi head học các loại quan hệ khác nhau:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (2.17)$$

Trong đó mỗi head được tính toán độc lập: $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

Ưu điểm của Multi-Head Attention:

- **Đa dạng hóa biểu diễn:** Mỗi head có thể học các loại quan hệ khác nhau (ví dụ: head 1 học quan hệ cú pháp, head 2 học quan hệ ngữ nghĩa, head 3 học quan hệ vị trí).

- **Tối ưu hóa song song trên GPU:** Việc chia thành nhiều head độc lập cho phép tận dụng tối đa khả năng tính toán song song của GPU. Thay vì thực hiện một phép attention lớn với chiều d_{model} , ta thực hiện h phép attention nhỏ với chiều $d_k = d_{model}/h$ song song. Điều này phù hợp hoàn hảo với kiến trúc SIMD (Single Instruction Multiple Data) của GPU, giúp:
 - Tăng tốc độ training đáng kể nhờ batch matrix multiplication
 - Tối ưu hóa throughput trong inference
 - Sử dụng hiệu quả băng thông bộ nhớ GPU (memory bandwidth)
 - Cho phép scale lên nhiều GPU dễ dàng hơn với tensor parallelism
- **Giảm chi phí tính toán:** Tổng chi phí tính toán tương đương với single-head attention có cùng chiều d_{model} , nhưng hiệu quả hơn nhờ song song hóa.

So sánh Transformer và CNN: Global Context vs Local Context

Một trong những điểm mấu chốt của Transformer so với CNN là cách nắm bắt ngữ cảnh:

Bảng 2.3: So sánh Transformer và CNN

Đặc điểm	Transformer	CNN
Ngữ cảnh	Global Context - Mỗi token có thể attention đến tất cả các token khác trong chuỗi	Local Context - Chỉ nhìn thấy các phần tử trong receptive field (kernel window)
Inductive Bias	Ít inductive bias → Cần lượng dữ liệu rất lớn để học	Có inductive bias mạnh về locality và translation equivariance
Dữ liệu cần	Yêu cầu dataset khổng lồ (hàng trăm GB văn bản)	Có thể hoạt động tốt với dataset nhỏ hơn
Phức tạp	$O(n^2)$ với độ dài chuỗi n	$O(n \cdot k^2)$ với kernel size k
Ưu điểm	Học được long-range dependencies tốt	Hiệu quả với dữ liệu có cấu trúc không gian (ảnh)

Inductive Bias: Transformer có rất ít giả định tiên nghiệm về cấu trúc dữ liệu (minimal inductive bias). Điều này có nghĩa là mô hình phải học toàn bộ ngữ cảnh và các pattern từ dữ liệu, thay vì có sẵn các “bias” như locality trong CNN. Hệ quả là Transformer cần lượng dữ liệu huấn luyện cực lớn để đạt được hiệu suất tốt.

Kiến trúc Encoder-Decoder và Decoder-Only

Transformer gốc bao gồm cả khối Encoder và Decoder:

- **Encoder:** Xử lý chuỗi đầu vào, tạo biểu diễn ngữ cảnh. Sử dụng bidirectional attention (nhìn cả trái và phải).
- **Decoder:** Sinh chuỗi đầu ra từng token một, sử dụng cross-attention để tham chiếu đến encoder output.

Tuy nhiên, các mô hình sinh văn bản như GPT chỉ sử dụng **Decoder-Only** architecture với cơ chế **Masked Self-Attention** (hay Causal Attention):

$$\text{Masked-Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + M \right) V \quad (2.18)$$

Trong đó M là ma trận mask với $M_{ij} = -\infty$ nếu $j > i$ (các vị trí tương lai bị che). Điều này đảm bảo:

- Mỗi token chỉ attend được đến các token trước nó (autoregressive)
- Tránh “học vẹt” (data leakage) - mô hình không thể nhìn thấy câu trả lời khi đang dự đoán
- Phù hợp cho các tác vụ sinh văn bản (text generation)

Các Khối Phụ trợ trong Transformer

- **Layer Normalization:** Khác với Batch Normalization trong CNN, Layer Norm chuẩn hóa theo chiều feature thay vì theo batch. Điều này phù hợp hơn cho sequence data có độ dài thay đổi:

$$\text{LayerNorm}(x) = \gamma \cdot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta \quad (2.19)$$

- **Residual Connection:** Skip connection giúp gradient flow tốt hơn và cho phép đào tạo các mạng rất sâu.
- **Feed-Forward Network (FFN):** Mỗi layer có một FFN với hidden dimension lớn (thường 4x embedding dimension):

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (2.20)$$

- **Positional Encoding:** Vì Transformer không có khái niệm về thứ tự, positional encoding được thêm vào để mã hóa vị trí của các token trong chuỗi.

b) Các mô hình LLM tiêu biểu và các bước ngoặt

Sự phát triển của LLM đã trải qua nhiều bước ngoặt quan trọng:

Các bước ngoặt trong lịch sử LLM:

1. **GPT-1 (2018):** OpenAI giới thiệu khái niệm pre-training + fine-tuning, chứng minh unsupervised pre-training có thể cải thiện NLP tasks.
2. **BERT (2018):** Google đề xuất bidirectional pre-training với masked language modeling, tạo bước đột phá trong NLU (Natural Language Understanding).
3. **GPT-3 (2020):** Với 175 tỷ tham số, GPT-3 chứng minh khả năng few-shot learning mà không cần fine-tuning, mở ra kỷ nguyên prompt engineering.
4. **ChatGPT (2022):** Sử dụng RLHF (Reinforcement Learning from Human Feedback) để align với mong muốn người dùng, tạo AI conversational thực sự hữu ích.
5. **GPT-4 (2023):** Mô hình đa phương thức (multimodal) có thể xử lý cả văn bản và hình ảnh, reasoning ability được cải thiện đáng kể.
6. **Open-source revolution (2023-2024):** LLaMA (Meta), Mistral, Qwen mở cửa cho cộng đồng nghiên cứu và triển khai local.
7. **DeepSeek (2024):** Bước ngoặt quan trọng từ Trung Quốc, DeepSeek-V3 đạt hiệu suất cạnh tranh với GPT-4 với chi phí huấn luyện thấp hơn đáng kể (chỉ \$5.6M), chứng minh hiệu quả của kiến trúc Mixture of Experts (MoE) và kỹ thuật Multi-head Latent Attention (MLA).

Bảng so sánh các mô hình LLM tiêu biểu:

Mô hình	Nhà phát triển	Tham số	Open-source	Đa phương thức	Đặc điểm nổi bật
GPT-4o	OpenAI	N/A	Không	Có	SOTA reasoning, API phổ biến
Gemini 2.0	Google	N/A	Không	Có	Tích hợp Google Search, dài context
Claude 3.5	Anthropic	N/A	Không	Có	An toàn, coding mạnh
LLaMA 3.1	Meta	8B-405B	Có	Hạn chế	Cộng đồng lớn, dễ fine-tune
Mistral Large	Mistral AI	7B-123B	Có	Có	Nhẹ, hiệu quả cho edge
Qwen 2.5	Alibaba	0.5B-72B	Có	Có	Đa ngôn ngữ, nhiều size
DeepSeek-V3	DeepSeek	671B MoE	Có	Có	Chi phí thấp, MoE hiệu quả

Xu hướng hiện tại trong LLM:

- **Mixture of Experts (MoE):** Chỉ kích hoạt một phần nhỏ tham số mỗi lần, giảm chi phí inference (DeepSeek, Mixtral).
- **Long context:** Mở rộng context window lên 128K-2M tokens (Gemini, Claude).
- **Multimodal:** Xử lý văn bản, hình ảnh, âm thanh, video trong cùng một mô hình.
- **Edge deployment:** Các mô hình nhỏ (1B-7B) chạy được trên smartphone và thiết bị nhúng.
- **Reasoning:** Cải thiện khả năng suy luận logic và toán học (o1, DeepSeek-R1).

c) Kỹ thuật Prompt Engineering

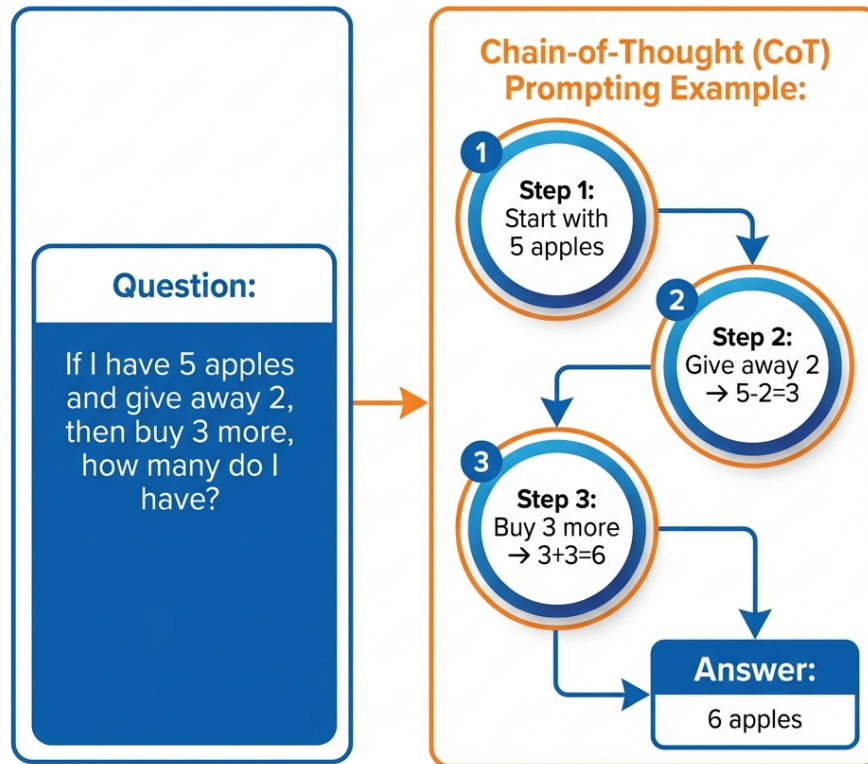
Prompt engineering là nghệ thuật thiết kế câu lệnh (prompt) để điều khiển hành vi của LLM. Các kỹ thuật prompting chính được so sánh trong bảng sau:

Kỹ thuật	Mô tả	Ví dụ prompt	Phù hợp với
Zero-shot	Yêu cầu trực tiếp không có ví dụ	“Dịch sang tiếng Anh: Xin chào”	Tác vụ đơn giản
Few-shot	Cung cấp 2-5 ví dụ mẫu trước khi hỏi	“Ví dụ: Mèo → Cat. Chó → Dog. Nhà → ?”	Tác vụ cần định dạng cụ thể
Chain-of-Thought (CoT)	Yêu cầu mô hình suy luận từng bước	“Hãy giải thích từng bước: $5 + 3 \times 2 = ?$ ”	Toán học, logic phức tạp
System Prompt	Định nghĩa vai trò và quy tắc cho agent	“Bạn là trợ lý y tế, luôn khuyên bệnh nhân gặp bác sĩ”	Chatbot, AI Agent
Self-consistency	Sinh nhiều câu trả lời, chọn đáp án phổ biến nhất	Chạy CoT nhiều lần, vote kết quả	Tăng độ chính xác

Ví dụ minh họa Chain-of-Thought Prompting:

Chain-of-Thought là kỹ thuật quan trọng giúp LLM giải quyết các bài toán phức tạp bằng cách yêu cầu suy luận từng bước.

Academic Diagram: Chain-of-Thought (CoT) Prompting Example



Hình 2.15: Ví dụ Chain-of-Thought: thay vì trả lời trực tiếp, mô hình giải thích từng bước suy luận để đi đến kết quả cuối cùng.

2.9.3 Retrieval-Augmented Generation (RAG)

RAG là kỹ thuật kết hợp LLM với hệ thống truy xuất thông tin, giúp mô hình trả lời dựa trên kiến thức cụ thể mà không cần fine-tune:

1. **Indexing:** Chuyển đổi tài liệu thành vector embeddings và lưu vào vector database
2. **Retrieval:** Khi nhận câu hỏi, tìm kiếm các đoạn văn bản liên quan nhất
3. **Generation:** LLM sinh câu trả lời dựa trên context được truy xuất

Trong ứng dụng chăm sóc người cao tuổi, RAG cho phép robot:

- Trả lời câu hỏi về lịch uống thuốc, lịch khám bệnh từ dữ liệu cá nhân
- Cung cấp thông tin y tế chính xác từ nguồn tin cậy

- Nhớ các cuộc hội thoại trước đó để tương tác cá nhân hóa

2.10 Hệ thống Nhận dạng Giọng nói và Xử lý Ngôn ngữ Tự nhiên

2.10.1 Một số giải pháp Nhận dạng Giọng nói:

Dưới đây là bảng so sánh các giải pháp chuyển đổi giọng nói thành văn bản (Speech-to-Text) phổ biến hiện nay:

Tiêu chí	Google Speech-to-Text	OpenAI Whisper	Vosk
Hình thức	Cloud API (SaaS)	Open-source (Local/Cloud)	Open-source (Local)
Độ chính xác	Rất cao (đặc biệt tiếng Việt)	Cực cao (State-of-the-art)	Khá (tùy thuộc model)
Internet	Bắt buộc	Không bắt buộc	Không cần thiết
Tốc độ	Phụ thuộc băng thông	Phụ thuộc GPU/CPU	Rất nhanh (Real-time)
Chi phí	Trả phí theo lưu lượng	Miễn phí phần mềm	Miễn phí
Phù hợp	Ứng dụng mobile cần độ chính xác tối đa.	Robot có phần cứng mạnh, cần hiểu ngữ cảnh sâu.	Thiết bị nhúng (Raspberry Pi), phản hồi cực nhanh.

Bảng 2.4: So sánh các công nghệ Speech-to-Text.

2.10.2 So sánh các phương pháp Xử lý Ngôn ngữ Tự nhiên (NLP)

Việc lựa chọn phương pháp NLP quyết định khả năng ”thông minh” và linh hoạt của Robot khi nhận lệnh.

Phương pháp	Đặc điểm	Độ phức tạp	Mở rộng	Ưu/Nhược điểm
Rule-based	Từ khóa và mẫu câu cố định.	Thấp	Kém	Ưu: Phản hồi tức thì. Nhược: Cứng nhắc.
Machine Learning	Phân loại ý định (Intent) qua TF-IDF, SVM.	Trung bình	Trung bình	Ưu: Hiểu biến thể nhẹ. Nhược: Cần dữ liệu.
Deep Learning	Transformer (BERT, GPT, Llama) hiểu sâu.	Cao	Rất cao	Ưu: Tự nhiên như người. Nhược: Cần GPU lớn.

Bảng 2.5: So sánh các phương pháp NLP.

2.10.3 Phương thức triển khai: Cloud Computing và Edge Computing

Để đảm bảo hệ thống hoạt động mượt mà trên phần cứng thực tế (Smartphone 1), tác giả lựa chọn giải pháp kết hợp giữa xử lý trực tiếp trên thiết bị và tận dụng tài nguyên máy chủ đám mây:

a. Cloud Computing Đây là phương án chủ đạo để xử lý các tác vụ nặng về tính toán và được ưu tiên sử dụng.

- **Cơ chế:** Robot gửi dữ liệu âm thanh hoặc văn bản lên các dịch vụ AI như Google Cloud hoặc OpenAI thông qua mạng Internet.
- **Ưu điểm:** Cho phép Robot trò chuyện thông minh, hiểu được các câu nói phức tạp nhờ sức mạnh xử lý cực mạnh của máy chủ mà không làm nóng điện thoại.
- **Lý do áp dụng:** Giúp khắc phục giới hạn về phần cứng, đảm bảo nội dung phản hồi từ Chatbot luôn nhanh chóng và chính xác.

b. Edge Computing Chỉ áp dụng cho các tác vụ xử lý NLP đơn giản

- **Cơ chế:** Sử dụng các thư viện nhẹ tích hợp sẵn để nhận diện các câu lệnh ngắn hay xử lý prompt
- **Hạn chế:** Chỉ xử lý được các mẫu câu đơn giản hay làm mượt các prompt hay phản hồi nhận được, không thể thực hiện các cuộc hội thoại linh hoạt nếu không có kết nối Internet.

2.11 Kết luận chương

Chương này đã trình bày đầy đủ cơ sở lý thuyết và một số phương pháp đề xuất cho bài toán phát hiện ngã sử dụng học sâu:

Về cơ sở lý thuyết:

- Giới thiệu nền tảng về mạng nơ-ron nhân tạo, từ cấu trúc neuron cơ bản đến các cơ chế lan truyền xuôi và lan truyền ngược
- Phân tích MLP và lý giải tại sao kiến trúc này không phù hợp trực tiếp cho xử lý ảnh/video
- Trình bày chi tiết CNN với các thành phần Convolution, Pooling, Fully Connected và các kiến trúc nổi tiếng như AlexNet, VGG, ResNet

- Giới thiệu YOLO như giải pháp real-time object detection, so sánh với các phương pháp two-stage truyền thống

Về lựa chọn mô hình:

- Phân tích lý do lựa chọn YOLOv11 dựa trên các tiêu chí: SOTA performance, cải tiến NMS, tốc độ inference, và phù hợp cho video giám sát
- Giải thích việc chọn phiên bản YOLOv11m như điểm cân bằng tối ưu giữa độ chính xác và tốc độ

Về xây dựng dataset:

- Giới thiệu dataset CauCaFall với ưu và nhược điểm
- Phân tích các khó khăn trong bài toán multi-task learning khi dataset không đồng nhất
- Đề xuất phương pháp xây dựng dataset mới bằng cách:
 - Bổ sung indoor objects annotation cho CauCaFall sử dụng pretrained YOLO
 - Chuyển đổi nhãn person trong COCO thành fall/no_fall sử dụng fine-tuned model
 - Hợp nhất và chuẩn hóa annotation theo format YOLO

Dataset multi-task được xây dựng với bộ nhãn cuối cùng gồm 8 classes, cho phép huấn luyện một mô hình duy nhất có khả năng phát hiện ngã và nhận diện vật thể trong nhà đồng thời. Điều này tạo nền tảng cho chương thực nghiệm tiếp theo, nơi sẽ trình bày chi tiết quá trình huấn luyện, đánh giá, và các kết quả đạt được của mô hình YOLOv11m trên dataset đề xuất.

Chương 3 – Phân tích và thiết kế hệ thống

3.1 Đề xuất thiết kế hệ thống

Với mục tiêu đặt ra cũng như những yêu cầu của đồ án thì hệ thống cần có một ứng dụng chạy trên nền tảng Android để làm khối điều khiển trung tâm của Robot với khả năng lấy dữ liệu camera, micro, của thiết bị từ đó tiến hành xử lý với thị giác máy tính để theo dõi, cảnh báo đến người thân khi người được giám sát bị té ngã hay với AI xử lý ngôn ngữ có thể trò chuyện một cách tự nhiên với người được chăm sóc. Ngoài ra ứng dụng này cũng cần có bộ phận xử lý các vấn đề về truyền nhận thông tin hay điều khiển Robot, thiết bị IoT tích hợp mở rộng.

Hệ thống cũng cần có một ứng dụng (dành cho người thân) để có thể giao tiếp từ xa với Robot, như nhận thông tin về cảnh báo té ngã, lập lịch nhắc nhở uống thuốc, gọi video hay thậm chí là điều khiển Robot từ xa để có thể tìm kiếm người thân xung quanh nhà. Mặc dù đây là ứng dụng mà đồ án không yêu cầu và có thể xem xét lược bỏ, tập trung xử lý ở khối điều khiển trung tâm của Robot nhưng với module này hệ thống sẽ hoàn thiện và thực tế hơn.

Với hai module trên thì hệ thống đã tương đối hoàn chỉnh tuy nhiên vẫn cần thêm các hệ thống hạ tầng trung gian để hỗ trợ xử lý chatbot, giọng nói hay cơ sở dữ liệu để lưu trữ thông tin, các giao thức và dịch vụ truyền thông, kết nối như MQTT hay Signaling Server...

Hệ thống được chia thành 3 khối chính: Khối Robot (Tại nhà), Hạ tầng Cloud (Trung gian) và Khối điều khiển (Từ xa).

Khối Robot (Tại nhà) Đây là thực thể vật lý tương tác trực tiếp với người cao tuổi, bao gồm hai phần chính:

- **Phần cứng Robot:** Đóng vai trò là khung thân vật lý, bao gồm các động cơ, cảm biến, và phần mạch kết nối điều khiển động cơ.
- **Smartphone 1 (Bộ não Robot):** Sử dụng một điện thoại Android làm trung tâm xử lý chính. Trên này chạy Ứng dụng Android Robot tích hợp các công nghệ AI quan trọng:
 - *AI Thị giác máy tính:* Thực hiện các tác vụ xử lý tại biên (Edge Computing) như phát hiện té ngã và theo dõi người dùng (tracking).

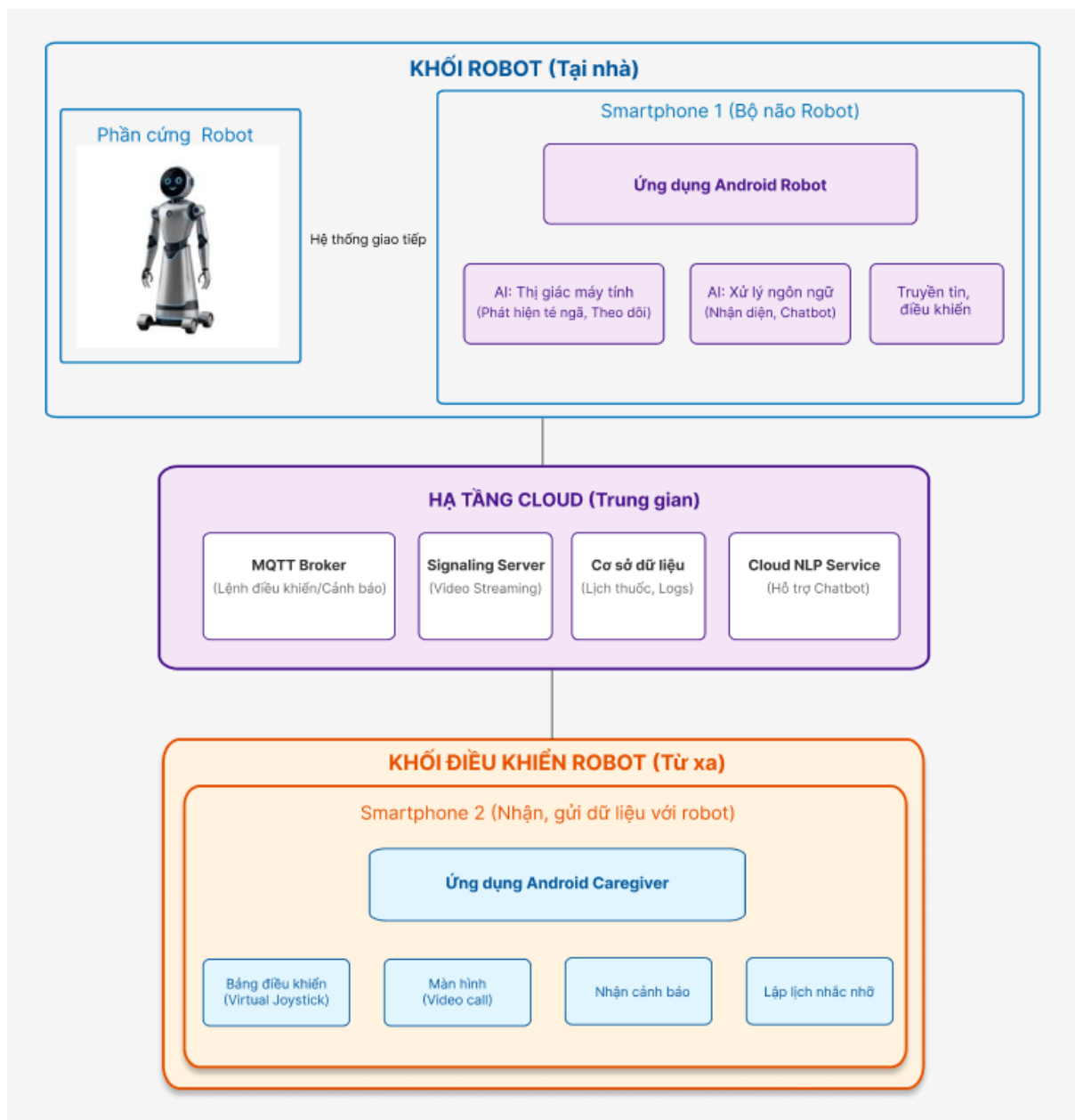
- *AI Xử lý ngôn ngữ*: Nhận dạng giọng nói và vận hành Chatbot để giao tiếp với người già.
- *Hệ thống truyền tin, điều khiển*: Là cầu nối trung gian để gửi lệnh điều khiển xuống phần cứng Robot, truyền thông tin liên quan camera, micro đến các khối liên quan.

Hạ tầng Cloud (Trung gian) Đóng vai trò là ”trạm trung chuyển” dữ liệu và cung cấp khả năng lưu trữ, hỗ trợ xử lý các tác vụ liên quan đến chatbot:

- **MQTT Broker**: Giao thức thông tin nhẹ, chuyên dùng để truyền các lệnh điều khiển từ người nhà đến robot hoặc gửi các cảnh báo khẩn cấp đến người nhà.
- **Signaling Server**: Phục vụ cho việc thiết lập kết nối truyền tải hình ảnh/video (Video Streaming) giữa Robot ứng dụng Caregiver
- **Cơ sở dữ liệu (Database)**: Lưu trữ các thông tin quan trọng như lịch uống thuốc, nhật ký hoạt động (logs) và dữ liệu người dùng.
- **Cloud NLP Service**: Hỗ trợ xử lý ngôn ngữ tự nhiên nâng cao cho Chatbot.

Khối Điều khiển Robot (Từ xa) Đây là giao diện dành cho người chăm sóc (Caregiver) để theo dõi và hỗ trợ người cao tuổi từ xa thông qua **Smartphone 2** chạy ứng dụng Android Caregiver:

- **Bảng điều khiển (Virtual Joystick)**: Điều khiển Robot di chuyển thủ công trong nhà.
- **Màn hình Video call**: Quan sát không gian xung quanh để có thể điều khiển Robot di chuyển và trò chuyện với người cao tuổi.
- **Nhận cảnh báo**: Nhận thông báo ngay lập tức nếu robot phát hiện người thân bị té ngã hoặc có bất thường.
- **Lập lịch nhắc nhở**: Cài đặt lịch uống thuốc hoặc các mốc thời gian quan trọng để robot nhắc nhở người cao tuổi tại nhà.



Hình 3.1: Thiết kế hệ thống đề xuất

3.2 Đề xuất giải pháp cho chức năng phát hiện té ngã

3.2.1 Lựa chọn YOLOv11 cho chức năng phát hiện té ngã

a) Thuộc nhóm State-of-the-Art (SOTA)

YOLOv11 đạt hiệu suất SOTA trên các benchmark chuẩn như COCO 2017 [11]. So với các phiên bản trước:

- Độ chính xác cao hơn YOLOv8 khoảng 1-2% mAP với cùng kích thước mô hình
- Số lượng parameters giảm 22% so với YOLOv8 (YOLOv11m: 20.1M vs YOLOv8m: 25.9M)
- Tốc độ inference nhanh hơn 10-15% nhờ tối ưu hóa module C3k2

Bảng so sánh benchmark với các thế hệ YOLO:

Model	Params (M)	mAP ₅₀₋₉₅	Latency (ms)	Năm
YOLOv5m	21.2	45.4%	4.0	2020
YOLOv7-M	36.9	51.4%	6.1	2022
YOLOv8m	25.9	50.2%	4.2	2023
YOLOv9m	20.0	51.4%	4.5	2024
YOLOv10m	16.5	51.3%	3.5	2024
YOLOv11m	20.1	51.5%	4.7	2024

Từ bảng benchmark, YOLOv11 cho thấy:

- Đạt mAP cao nhất (51.5%) trong các phiên bản YOLO
- Hiệu quả parameters tốt hơn YOLOv8 (ít hơn 22% params nhưng mAP cao hơn)
- Cân bằng tốt giữa accuracy và speed

b) Cải tiến về Non-Maximum Suppression (NMS)

Các phiên bản YOLO truyền thống sử dụng NMS như bước hậu xử lý để loại bỏ các bounding box trùng lặp. NMS có nhược điểm:

- Là thao tác tuần tự, không song song hóa được trên GPU
- Tạo bottleneck trong quá trình inference
- Cần tinh chỉnh ngưỡng IoU, ảnh hưởng đến kết quả

YOLOv11 tích hợp các cơ chế giảm phụ thuộc vào NMS thông qua loss function và kiến trúc head được thiết kế để giảm thiểu các prediction trùng lặp ngay từ đầu.

c) Tốc độ suy luận nhanh, phù hợp thời gian thực

Trong ứng dụng phát hiện ngã, thời gian phản hồi là yếu tố sống còn. YOLOv11 đạt:

- Hàng trăm FPS trên GPU hiện đại (RTX 3060 trở lên)
- Có thể chạy trên thiết bị edge như Jetson Nano với tốc độ chấp nhận được
- Latency thấp, phù hợp cho hệ thống giám sát cần cảnh báo tức thì

d) Hiệu quả trên video giám sát

Video giám sát trong nhà thường có đặc điểm:

- Góc camera cố định, background ít thay đổi
- Ánh sáng không đều, có thể thiếu sáng
- Đối tượng (người) chiếm phần lớn khung hình

YOLOv11 được huấn luyện với data augmentation mạnh mẽ (mosaic, mixup, photometric distortion), giúp mô hình robust với các điều kiện này.

3.2.2 Lựa chọn YOLOv11n cho Robot

Đồ án này lựa chọn **YOLOv11n** (nano) thay vì các phiên bản lớn hơn vì các lý do sau:

- **Phù hợp với nền tảng Robot:** Robot sử dụng smartphone làm bộ xử lý chính, YOLOv11n với chỉ 2.6M parameters có thể chạy mượt mà trên điện thoại Android
- **Real-time trên thiết bị nhúng:** Đạt 25-45 FPS trên smartphone với TensorFlow Lite, đủ cho giám sát thời gian thực
- **Tiết kiệm pin:** Model nhẹ tiêu thụ ít năng lượng hơn, quan trọng cho robot di động sử dụng pin
- **Độ chính xác chấp nhận được:** mAP 39.5% đủ tốt cho bài toán Fall Detection (chỉ cần phân biệt fall/no_fall)

- **Latency thấp:** 1.55ms cho phép phản hồi nhanh trong trường hợp khẩn cấp
- **Dễ fine-tune:** Model nhỏ huấn luyện nhanh hơn trên GPU tầm trung, ít overfitting hơn trên dataset nhỏ như CauCaFall

Ước tính FPS của YOLOv11n trên các nền tảng:

Nền tảng	FPS ước tính	Ghi chú
GPU NVIDIA (TensorRT)	400-600	RTX 3060+
GPU NVIDIA (PyTorch)	150-250	RTX 2060+
Apple CoreML	40-100	iPhone 12+
Android (TFLite)	25-45	Snapdragon 8xx
CPU (ONNX)	15-20	Intel i7
Jetson Nano	10-15	Edge deployment

Ghi chú: FPS thực tế phụ thuộc vào kích thước ảnh đầu vào (mặc định 640x640) và điều kiện phần cứng.

3.3 Đề xuất giải pháp cho chức năng theo dõi(tracking)

3.3.1 Các thuật toán Tracking phổ biến

Trong lĩnh vực Multi-Object Tracking (MOT), có nhiều thuật toán đã được phát triển. Bảng sau so sánh các thuật toán tracking phổ biến:

Bảng 3.1: So sánh các thuật toán Multi-Object Tracking

Thuật toán	Motion	Appearance	MOTA	FPS	ReID	Năm
SORT	Kalman Filter	Không	74.6	260	Không	2016
DeepSORT	Kalman Filter	Deep CNN	75.4	20	Có	2017
ByteTrack	Kalman Filter	Không	80.3	30	Không	2022
OC-SORT	Kalman + ORU	Không	78.0	28	Không	2022
BoT-SORT	Kalman + CMC	BoT features	80.5	15	Có	2022

Benchmark trên MOT17 test set

Đề án này lựa chọn **ByteTrack** [5] làm thuật toán tracking chính vì đạt cân bằng tốt nhất giữa độ chính xác và tốc độ, đồng thời không yêu cầu mạng Re-identification phụ.

3.3.2 Chi tiết thuật toán ByteTrack

ByteTrack [5] được đề xuất bởi Zhang et al. (ECCV 2022) với ý tưởng đột phá: *tận dụng cả các detection có confidence thấp* thay vì loại bỏ chúng như các tracker truyền thống.

Vấn đề với các tracker truyền thống:

- Các tracker như SORT, DeepSORT chỉ sử dụng detection có confidence cao (thường > 0.5)
- Detection thấp bị loại bỏ, gây mất track khi đối tượng bị che khuất một phần
- Dẫn đến ID switch và fragmentation cao

Ý tưởng cốt lõi của ByteTrack:

“Every detection box, including the low score ones, is a true positive if they match correctly.”

Thuật toán BYTE (2-stage association):

ByteTrack thực hiện association hai giai đoạn:

1. Giai đoạn 1 - High score matching:

- Lọc các detection có score $\geq \tau_{high}$ (thường $\tau_{high} = 0.6$)
- Dùng Kalman Filter dự đoán vị trí các track hiện có
- Ghép nối high-score detections với tracks bằng Hungarian Algorithm + IoU
- Các track không match được chuyển sang giai đoạn 2

2. Giai đoạn 2 - Low score matching:

- Lọc các detection có score trong khoảng $[\tau_{low}, \tau_{high})$ (thường $\tau_{low} = 0.1$)
- Ghép nối low-score detections với *unmatched tracks* từ giai đoạn 1
- Điều này giúp duy trì track khi đối tượng bị occluded (detection score thấp)

Pseudo-code ByteTrack:

Input: Detections D , Tracks T , thresholds τ_{high} , τ_{low}

Output: Updated Tracks T'

```
1.  $D_{\text{high}} = \{d \in D \mid d.\text{score} \geq \tau_{\text{high}}\}$ 
2.  $D_{\text{low}} = \{d \in D \mid \tau_{\text{low}} \leq d.\text{score} < \tau_{\text{high}}\}$ 

# Stage 1: Match high-score detections
3. Predict track positions using Kalman Filter
4. Compute IoU cost matrix between  $D_{\text{high}}$  and  $T$ 
5. Match using Hungarian Algorithm
6.  $T_{\text{matched}}, T_{\text{unmatched}}, D_{\text{unmatched}} = \text{Assignment}$ 

# Stage 2: Match low-score detections
7. Compute IoU cost matrix between  $D_{\text{low}}$  and  $T_{\text{unmatched}}$ 
8. Match using Hungarian Algorithm
9. Update  $T_{\text{matched}}$  with matched low-score detections

# Post-processing
10. Initialize new tracks from  $D_{\text{unmatched}}$  (high-score)
11. Mark  $T_{\text{unmatched}}$  as lost (delete if lost > max_age)
```

Lý do chọn ByteTrack cho hệ thống Fall Detection:

1. **Cân bằng tốc độ-chính xác:** MOTA 80.3% với 30 FPS, đủ cho real-time trên smart-phone
2. **Không cần ReID network:** Giảm độ phức tạp và tài nguyên tính toán
3. **Xử lý occlusion tốt:** Low-score matching giúp duy trì track khi người ngã (confidence thường giảm do tư thế bất thường)
4. **Đơn giản:** Chỉ cần YOLOv11 detection + ByteTrack, không cần model phụ
5. **ID switch thấp:** Chỉ 159 ID switches trên MOT17, quan trọng cho việc theo dõi liên tục người cao tuổi

3.3.3 Person Re-identification (ReID)

Person Re-identification (ReID) là bài toán nhận diện lại cùng một người từ các camera khác nhau hoặc sau khi người đó ra khỏi và quay lại khung hình. Đây là thành phần quan trọng để duy trì identity người dùng trong hệ thống giám sát đa camera.

Kiến trúc hệ thống ReID:

1. Feature Extraction (Trích xuất đặc trưng):

- Backbone CNN (ResNet-50, OSNet) trích xuất visual features từ ảnh người
- Output: Embedding vector 256-2048 chiều đại diện cho appearance
- Có thể bổ sung local features từ các phần cơ thể (head, torso, legs)

2. Metric Learning (Học khoảng cách):

- **Triplet Loss:** Kéo gần các ảnh cùng người, đẩy xa các ảnh khác người

$$L_{triplet} = \max(0, d(a, p) - d(a, n) + \alpha) \quad (3.1)$$

với a : anchor, p : positive, n : negative, α : margin

- **Contrastive Loss:** So sánh từng cặp ảnh positive/negative
- **Center Loss:** Giảm intra-class variation

3. Gallery Matching (So khớp):

- Query image được so sánh với gallery database chứa các ID đã biết
- Cosine similarity hoặc Euclidean distance để ranking
- Re-ranking techniques để cải thiện kết quả

Bảng so sánh các mô hình ReID phổ biến:

Bảng 3.2: So sánh các mô hình Person Re-identification

Mô hình	Backbone	mAP	FPS	Đặc điểm
OSNet	Custom lightweight	84.9%	100+	Nhẹ, phù hợp real-time
AGW	ResNet-50 + Attention	87.8%	50	Global-local features
TransReID	ViT + Side info	89.5%	30	SOTA accuracy
FastReID	ResNet-50	86.4%	80	Easy deployment

Benchmark trên Market-1501 dataset

3.4 Đề xuất giải pháp cho chức năng xử lý ngôn ngữ, giao tiếp

Quy trình xử lý ngôn ngữ tự nhiên (NLP Pipeline) Để robot có khả năng nghe - hiểu và phản hồi tự nhiên, hệ thống triển khai quy trình ba bước:

- **Nhận dạng giọng nói (ASR - Automatic Speech Recognition):** Sử dụng *Google Speech-to-Text API* tích hợp trực tiếp trên Smartphone 1. Công nghệ này hỗ trợ tốt tiếng Việt đa vùng miền và có khả năng xử lý tốt trong môi trường có tiếng ồn nền.
- **Xử lý hội thoại (Cloud NLP Service):** Sau khi chuyển âm thanh thành văn bản, dữ liệu được gửi lên Cloud để xử lý thông qua API của các mô hình ngôn ngữ lớn (LLM) như *OpenAI* hoặc *Google Cloud*. Lựa chọn này giúp robot có khả năng hiểu ngữ cảnh sâu, đưa ra lời khuyên về sức khỏe hoặc trò chuyện tâm sự với người già thay vì chỉ phản hồi theo kịch bản cứng nhắc.
- **Tổng hợp giọng nói (TTS - Text-to-Speech):** Sử dụng *Google TTS* hoặc *FPT.AI* để chuyển văn bản phản hồi thành giọng nói. Ưu tiên sử dụng các giọng đọc vùng miền (Bắc/Trung/Nam) tự nhiên để tạo cảm giác gần gũi cho người dùng.

3.5 Đề xuất các giao thức giao tiếp sử dụng

Trong quá trình nghiên cứu và phát triển hệ thống, một số giao thức truyền giao tiếp được sử dụng để giải quyết các vấn đề kết nối, giao tiếp giữa các module trong hệ thống.

3.5.1 Giao thức MQTT (Message Queuing Telemetry Transport)

Đây là giao thức chủ đạo được sử dụng để truyền tải dữ liệu điều khiển và cảnh báo thông qua MQTT Broker trung gian.

- **Ứng dụng:** Truyền lệnh điều khiển từ giao diện bảng điều khiển (Virtual Joystick) trên thiết bị của người thân đến Robot. Đồng thời, Robot sử dụng giao thức này để gửi các cảnh báo khẩn cấp (như phát hiện té ngã) về ứng dụng giám sát.
- **Ưu điểm:** Tiết kiệm băng thông, tiêu tốn ít năng lượng, hoạt động hiệu quả trong điều kiện mạng không ổn định, đặc biệt phù hợp cho các thiết bị IoT và di động.

3.5.2 Giao thức WebRTC (Real-Time Communication)

Giao thức này được triển khai thông qua Signaling Server để thiết lập các kênh truyền thông đa phương tiện thời gian thực.

- **Ứng dụng:** Hỗ trợ tính năng truyền hình ảnh trực tiếp (Video Streaming) và đàm thoại video (Video Call), cho phép người thân giám sát trực tiếp qua camera của robot với độ trễ cực thấp.
- **Cơ chế:** Sử dụng Signaling Server để thực hiện quá trình bắt tay (handshake) giữa hai thiết bị Android, sau đó dữ liệu video được truyền trực tiếp theo mô hình ngang hàng (Peer-to-Peer) để tối ưu hóa tốc độ.

3.5.3 Giao thức HTTP/HTTPS

Được sử dụng làm cầu nối giao tiếp chính với hệ thống cơ sở dữ liệu và các dịch vụ tính toán đám mây.

- **Ứng dụng:**
 - Đồng bộ hóa dữ liệu về lịch nhắc nhở y tế, nhật ký hoạt động (Logs) của người cao tuổi.
 - Gửi yêu cầu xử lý ngôn ngữ tự nhiên (NLP) lên Cloud để nhận phản hồi thông minh từ hệ thống Chatbot.
- **Ưu điểm:** Khả năng tương thích cao với các dịch vụ đám mây hiện đại, đảm bảo tính toàn vẹn và an toàn dữ liệu thông qua mã hóa SSL/TLS.

3.5.4 Giao tiếp nội bộ (Serial/Bluetooth)

Tại Robot, việc truyền tín hiệu giữa Smartphone đóng vai trò "bộ não" và phần cứng điều khiển động cơ được thực hiện qua các chuẩn kết nối tầm ngắn.

- **Ứng dụng:** Thực hiện việc giao tiếp giữa Smartphone và phần cứng Robot.
- **Chuẩn kết nối:** Ưu tiên sử dụng giao tiếp Serial (UART) thông qua cổng kết nối vật lý giúp đảm bảo tin cậy và tốc độ cao hoặc giao thức Bluetooth.

3.5.5 Kết nối Internet (TCP/IP)

Toàn bộ các thành phần trong hệ thống, bao gồm Robot (Smartphone 1) và ứng dụng người thân (Smartphone 2), luôn duy trì kết nối thông qua giao thức TCP/IP trên nền tảng mạng Wi-Fi hoặc 4G/5G. Đây là nền tảng hạ tầng cốt lõi đảm bảo khả năng kết nối xuyên suốt và đồng bộ dữ liệu với các dịch vụ trên Cloud. Một số ưu và nhược điểm của hai phương thức kết nối như sau:

- **Mạng không dây Wi-Fi:**

- *Ưu điểm:* Cung cấp băng thông lớn và ổn định, đặc biệt quan trọng cho việc truyền tải luồng dữ liệu đa phương tiện (Video Call/Streaming) qua WebRTC với độ phân giải cao. Chi phí vận hành thấp và giúp tiết kiệm điện năng cho Smartphone hơn so với mạng di động.
- *Nhược điểm:* Phạm vi phủ sóng bị hạn chế trong không gian gia đình, tín hiệu có thể bị suy giảm khi Robot di chuyển qua các vật cản như tường hoặc cửa.

- **Mạng di động (4G/5G):**

- *Ưu điểm:* Đảm bảo tính linh động tối đa cho ứng dụng người thân (Caregiver App), cho phép giám sát và nhận cảnh báo té ngã từ Robot ở bất cứ đâu có sóng di động. 5G mang lại ưu thế về độ trễ cực thấp, hỗ trợ tốt cho việc điều khiển Robot qua MQTT thời gian thực.
- *Nhược điểm:* Độ trễ (Latency) và độ biến thiên trễ có thể thay đổi tùy thuộc vào khu vực và nhà mạng, ảnh hưởng đến sự đồng bộ của video. Ngoài ra, việc sử dụng liên tục sẽ tiêu tốn nhiều năng lượng và phát sinh chi phí dữ liệu di động.

Trong ứng dụng thực tế Robot có thể ưu tiên sử dụng mạng di động để đảm bảo tính tin cậy khi có sự cố mất điện tuy nhiên một số khác lại ưu tiên vấn đề chi phí hay tiêu tốn pin. Còn trong bối cảnh đề án này nên ưu tiên sử dụng kết nối Wifi để triển khai dễ dàng, tiết kiệm hơn.

3.6 Vấn đề liên quan mất đồng bộ giữa tín hiệu điều khiển Robot và Video call

Tín hiệu điều khiển robot qua MQTT và video robot truyền về người điều khiển qua WebRTC có tốc độ khác nhau, gây ra vấn đề mất đồng bộ. Phần này đề xuất các giải pháp đảm bảo hệ thống chạy ổn định.

1. Vấn đề mất đồng bộ:

Thách thức về sự mất đồng bộ nảy sinh khi có sự sai lệch về thời gian truyền dẫn giữa giao thức MQTT (điều khiển) và WebRTC (video). Sự mất cân bằng này tạo ra ”độ trễ thị giác”, khiến người điều khiển không thể nắm bắt chính xác trạng thái thực tế của Robot trong không gian, từ đó gia tăng rủi ro hư hại thiết bị do va đập. Chính vì thế, việc kiểm soát và đồng bộ hóa các luồng tín hiệu là một nhiệm vụ quan trọng để nâng cao hiệu quả vận hành từ xa.

- **MQTT (TCP):** Tín hiệu điều khiển đi qua Cloud broker
- **WebRTC (UDP):** Video streaming P2P
- Hai luồng dữ liệu độc lập với tốc độ và đường truyền khác nhau

2. Giải pháp Timestamp và Cảnh báo Kết nối:

Mỗi gói tin điều khiển gửi qua MQTT và mỗi khung hình WebRTC đều được gắn nhãn thời gian (timestamp). Khi ứng dụng người thân nhận được, nó sẽ tính toán độ lệch trễ:

$$\Delta t = t_{receive} - t_{send} \quad (3.2)$$

Trong đó:

- t_{send} : Thời điểm gửi gói tin từ robot
- $t_{receive}$: Thời điểm nhận được tại ứng dụng người thân
- Δt : Độ trễ thực tế của kết nối

Cơ chế cảnh báo:

- Nếu $\Delta t < 100ms$: Kết nối tốt, điều khiển bình thường
- Nếu $100ms \leq \Delta t < 300ms$: Kết nối trung bình, hiển thị cảnh báo nhẹ
- Nếu $\Delta t \geq 300ms$: Cảnh báo “*Kết nối yếu*” để người dùng giảm tốc độ điều khiển
- Các ngưỡng thời gian đưa ra cảnh báo có thể thay đổi khi tiến hành chạy thực nghiệm

3. Giải pháp Cảm biến Phần cứng (Collision Avoidance):

Để đảm bảo an toàn khi có độ trễ lớn, khối phần cứng Robot được tích hợp cảm biến siêu âm hoặc hồng ngoại để tự động tránh vật cản:

- **Cảm biến siêu âm:** thường đo khoảng cách từ 3cm đến 400cm
- **Cảm biến hồng ngoại:** thường phát hiện vật cản ở khoảng cách gần ($<30\text{cm}$)

Một số ưu nhược điểm của từng cảm biến:

- **Cảm biến siêu âm (Ultrasonic Sensor):**

- *Ưu điểm:* Có phạm vi đo đặc rộng (lên đến 400cm), cho phép Robot nhận biết vật cản từ xa để giảm tốc độ dần đều thay vì dừng đột ngột. Đặc biệt, cảm biến không bị ảnh hưởng bởi màu sắc hay độ trong suốt của vật cản (như cửa kính).
- *Nhược điểm:* Tốc độ phản hồi phụ thuộc vào tốc độ âm thanh nên chậm hơn so với cảm biến ánh sáng. Có thể xảy ra hiện tượng "góc chết" khi gặp các bề mặt không phẳng gây tán xạ âm hoặc các bề mặt hấp thụ âm như rèm cửa, thảm trải sàn.

- **Cảm biến hồng ngoại (Infrared Sensor):**

- *Ưu điểm:* Kích thước nhỏ gọn, chi phí thấp và tốc độ phản hồi cực nhanh do sử dụng tín hiệu ánh sáng. Đây là giải pháp tối ưu cho việc phát hiện vật cản ở cự ly gần để kích hoạt dừng khẩn cấp.
- *Nhược điểm:* Phạm vi hoạt động hẹp ($<30\text{-}80\text{cm}$). Độ chính xác bị ảnh hưởng nghiêm trọng bởi ánh sáng môi trường (ánh nắng mặt trời) và màu sắc của vật thể; các vật thể màu đen có thể hấp thụ tia hồng ngoại dẫn đến việc cảm biến không phát hiện được vật cản.

Bên cạnh các cảm biến tiệm cận truyền thống, công nghệ LiDAR (Light Detection and Ranging) được xem xét như một giải pháp nâng cao nhằm hỗ trợ khả năng tự hành của Robot. Bằng cách sử dụng các xung laser để quét môi trường xung quanh và xây dựng bản đồ điểm, LiDAR cho phép Robot thực hiện các thuật toán định vị và lập bản đồ đồng thời. Điều này giúp gia tăng độ chính xác trong việc di chuyển và hỗ trợ người cao tuổi trong không gian phức tạp, tuy nhiên đòi hỏi sự đầu tư lớn hơn về chi phí và tài

nguyên tính toán của khối xử lý trung tâm. vậy nên với bối cảnh đề án vẫn nên sử dụng các giải pháp đơn giản ở trên, LiDAR là một công nghệ cân nhắc cho sự phát triển hoàn chỉnh sau này với các dự án thực tế.



Chương 4 – Hiện thực hệ thống



Chương 5 – Kết quả và thảo luận

Chương 6 – Kết luận

6.1 Nhận xét

6.2 Hướng phát triển

TÀI LIỆU THAM KHẢO

1. J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779-788. [Online]. Available: <https://arxiv.org/abs/1506.02640>
2. A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention Is All You Need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. [Online]. Available: <https://arxiv.org/abs/1706.03762>
3. K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770-778. [Online]. Available: <https://arxiv.org/abs/1512.03385>
4. N. Wojke, A. Bewley, and D. Paulus, “Simple Online and Realtime Tracking with a Deep Association Metric,” in *IEEE International Conference on Image Processing (ICIP)*, 2017, pp. 3645-3649. [Online]. Available: <https://arxiv.org/abs/1703.07402>
5. Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *European Conference on Computer Vision (ECCV)*, 2022. [Online]. Available: <https://arxiv.org/abs/2110.06864>
6. M. Müller et al., “OpenBot: Turning Smartphones into Robots,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2021. [Online]. Available: <https://www.openbot.org/>
7. T.-Y. Lin et al., “Microsoft COCO: Common Objects in Context,” in *European Conference on Computer Vision (ECCV)*, 2014, pp. 740-755. [Online]. Available: <https://arxiv.org/abs/1405.0312>
8. A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2012, pp. 1097-1105.

9. K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *International Conference on Learning Representations (ICLR)*, 2015. [Online]. Available: <https://arxiv.org/abs/1409.1556>
10. R. Khanam and M. Hussain, “YOLOv11: An Overview of the Key Architectural Enhancements,” *arXiv preprint arXiv:2410.17725*, 2024. [Online]. Available: <https://arxiv.org/abs/2410.17725>
11. H. Khan, I. Ullah, M. Shabaz, M. F. Omer, M. T. Usman, M. S. Guellil, and J. Koo, “Visionary vigilance: Optimized YOLOV8 for fallen person detection with large-scale benchmark dataset,” *Image and Vision Computing*, vol. 149, 2024, Art. no. 105195. [Online]. Available: <https://doi.org/10.1016/j.imavis.2024.105195>
12. Z. Zou, K. Chen, Z. Shi, Y. Guo, and J. Ye, “Object Detection in 20 Years: A Survey,” *Proceedings of the IEEE*, vol. 111, no. 3, pp. 257-276, Mar. 2023. [Online]. Available: <https://arxiv.org/abs/1905.05055>